KISS Sorcar: A Stupidly-Simple General-Purpose and Software Engineering AI Assistant

Koushik Sen
EECS Department, UC Berkeley
ksen@berkeley.edu

"Everything should be made as simple as possible, but not simpler." — Albert Einstein

Abstract

Large language models can generate code and call tools fluently, yet deploying them as practical assistants for *long-horizon* software engineering and AI-discovery tasks still exposes persistent gaps: finite context windows, a single mistake that can derail entire sessions, agents that get stuck in dead ends, AI slop, and generated changes that are difficult to review or revert.

We present **KISS Sorcar**, an *open-source general-purpose AI agent for long-horizon tasks and AI discovery* that doubles as an integrated development environment (IDE). It is built on top of the **KISS Agent Framework**, a stupidly-simple AI agent framework of roughly 2,900 lines of code for the core agents. The framework addresses the gaps above through a structured system prompt and a five-layer agent hierarchy in which each layer adds exactly one concern: budget-tracked ReAct execution, automatic continuation across sub-sessions via summarization, coding and browser tools with parallel sub-agents, persistent multi-turn chat with history recall, and git worktree isolation so every task runs on its own branch. Engineering principles are encoded in the agent's system prompt.

KISS Sorcar is a free, simple, local-first, bring-your-own-key assistant that ships as three coordinated surfaces over a single local daemon: a Visual Studio Code extension, a Claude-Code-style CLI (`sorcar`), and a browser/mobile web app; all agents run as daemons. Prompts and code are sent directly to the model provider or local endpoint the user configures, not through any intermediary servers. The framework supports mixing models from multiple vendors in the same task simply via prompts—OpenAI, Anthropic, Gemini, Together, Z.AI, Moonshot AI, OpenRouter, Claude Code CLI, and Codex CLI—bundles a catalog of 504 models across 9 provider categories, includes 23 third-party messaging agents (Slack, Gmail, WhatsApp, SMS, phone-call control, and others), discovers Model-Context-Protocol (MCP) servers, loads Agent Skills, and supports browser automation, multimodal input, and Docker containers.

In this research, we deliberately prioritize output quality over speed: giving a frontier model adequate time to validate its own output—running linters, type checkers, and tests—reduces the low-quality code common in faster but less thorough agents. The entire system was built using itself in 4 months, providing a continuous stress test in which any bug was patched as it appeared. On Terminal Bench 2.0, KISS Sorcar achieves a 62.2% overall pass rate with Claude Opus 4.6, compared with Claude Code (58%) and Cursor Composer 2 (61.7%). These results are notable because we did not tune our prompts or any model specifically for Terminal Bench 2.0.

Preprint.

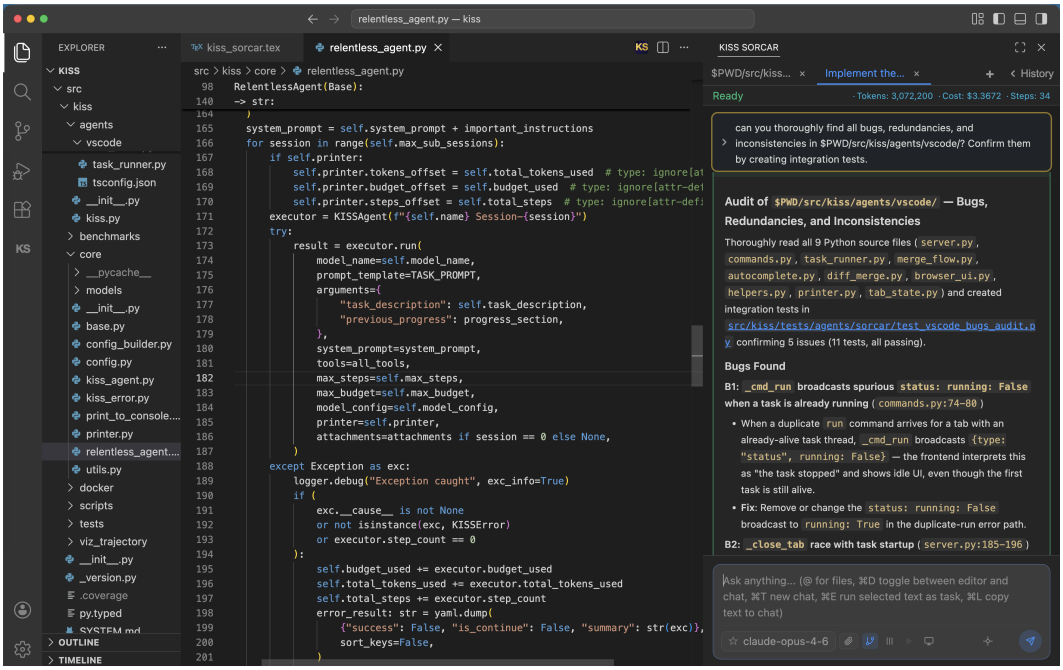

Figure 1: Screenshot of KISS Sorcar running as a VS Code extension. The sidebar shows the agent's chat interface with real-time budget tracking, while the editor displays the code being modified.

1 Introduction

Modern Large language models (LLMs), such as Anthropic's Claude Opus 4.7 [Anthropic, 2026b], OpenAI's GPT 5.5 [OpenAI, 2026b], and Google's Gemini 3.1 [Google DeepMind, 2026], can generate code, reason about software architecture, and use developer tools [Chen et al., 2021, Rozière et al., 2023]. A growing body of work has explored how to harness these capabilities for autonomous software engineering, from single-session agents that resolve GitHub issues [Yang et al., 2024b, Wang et al., 2024b] to industrial products marketed as AI software and general assistants [GitHub, 2021, Cursor, 2024, Cognition Labs, 2024, Anthropic, 2025b, OpenAI, 2025a, OpenClaw AI, 2025]. Yet using an LLM as a practical *general-purpose* assistant for long-horizon software engineering and AI-discovery tasks still exposes several stubborn gaps: context windows are finite, a single mistake can derail an entire session, agents get stuck in dead ends, models generate AI slop, and generated changes are difficult to review or revert once applied to a live codebase.

We propose the *KISS Agent Framework*, a stupidly simple AI agent framework containing roughly 2,900 lines of code for the core agent implementation. We try to address the above-mentioned gaps through a structured system prompt (Section 6) and a five-layer agent hierarchy in which each layer solves exactly one concern:

1. **KISS Agent** — budget-tracked ReAct [Yao et al., 2023b] loop with native function calling.
2. **Relentless Agent** — automatic summarization and continuation across sub-sessions.
3. **Sorcar Agent** — coding tools, browser automation, and parallel sub-agent execution.
4. **Chat Sorcar Agent** — persistent multi-turn chat sessions with history recall.
5. **Worktree Sorcar Agent** — git worktree isolation so every task runs on its own branch.

The name "KISS" reflects the *Keep It Simple, Stupid* design philosophy in software engineering: each layer is small, each concern is isolated, and the overall system avoids unnecessary abstraction.

Table 1 shows a high-level comparison of KISS Sorcar with Claude Code and Cursor.

Table 1: KISS Sorcar compared with Claude Code and Cursor, reproduced from the project README.

Capability	KISS Sorcar	Claude Code	Cursor
Interfaces	CLI + VS Code extension + web/mobile app	CLI + mobile app	Custom VS Code
AI Discovery	✓ simply via prompt	×	×
GEPA Prompt Optimization	✓ simply via prompt	×	×
Multiple models from multiple vendors in the same task	✓ Mix OpenAI, Anthropic, Gemini, Together, Z.AI, Moonshot AI, OpenRouter, Claude Code CLI, and Codex CLI	× Anthropic Claude models only	× One model per task
Primary focus	✓ **Quality**—rigorous review, end-to-end tests	Speed and developer ergonomics	Speed
Core Agents # LoC	**~2900**	Unknown	Unknown
Models in bundled catalog	504 across 9 provider categories	Claude family only	Subset chosen by Cursor
Bring your own API key / endpoint	✓ Yes—keys stay on your machine	✓ Anthropic key	△ Routed through Cursor backend
Open source	✓ Apache-2.0	× Proprietary	× Proprietary
Price	Free framework; pay only your chosen model provider	Subscription / API usage	Subscription
Run on top of Claude Code / Codex CLI	✓ `cc/*` and `codex/*` namespaces	N/A	×
Messaging and communication channels	✓ 23 third-party agents, including Slack, Gmail, Phone Control, SMS, and WhatsApp	Partial: Slack, mobile Remote Control, and research-preview channels for Telegram, Discord, and iMessage; no documented built-in Gmail, WhatsApp, phone-call, or SMS channel	Partial: Slack and Microsoft Teams Cloud Agent integrations; no documented built-in Gmail, WhatsApp, phone-call, or SMS channel
Terminal Bench 2.0 score	**62.2%**	58%	61.7% (Cursor agent)

We implement KISS Sorcar, an open-source general-purpose AI agent for long-horizon tasks and AI discovery, which doubles as an integrated development environment (IDE) on top of the KISS Agent Framework. KISS Sorcar is local-first and bring-your-own-key: prompts and code are sent directly to the model provider or local endpoint the user configures, not through any intermediary servers. It ships as three coordinated surfaces over a single local daemon—a Visual Studio Code extension, a Claude-Code-style command-line interface (`sorcar`), and a browser/mobile web app—all running locally; all agents run as daemons. The CLI has both an interactive REPL mode (with slash commands such as `/help`, `/clear`, `/resume`, `/model`, `/cost`, `/skills`, `/mcp`, `/autocommit`, and file/folder `@`-mentions) and a one-shot non-interactive mode (`-t`/`-f`). KISS Sorcar has browser support (using open-source Chromium and Playwright), multimodal support, Docker container support, OpenClaw-like features (whose discussion is beyond the scope of the paper), a mobile/web app, integrated MCP server management (`sorcar mcp add/list/auth/debug/...`), and a library of Agent Skills loaded from `~/.kiss/skills` and project-local directories. KISS Sorcar is free and open-source (Apache-2.0), distributed on PyPI as `kiss-agent-framework`; all one needs is a model API key from a major LLM provider, such as Anthropic, OpenAI, Google, Together, Z.AI, Moonshot AI, or OpenRouter, or a custom local endpoint configured via `-endpoint`/`-header`. The framework can also run on top of the Claude Code CLI (`cc/*` models) and the Codex CLI (`codex/*` models), and mix models from multiple vendors within a single task simply by issuing prompts. We implemented this framework in roughly 4 months, and the repository is available at `https://github.com/ksenxx/kiss_ai`. The name "Sorcar" pays homage to P. C. Sorcar, the Bengali magician. The engineering principles described in Section 6 are encoded in the agent's system prompt.

KISS Sorcar has been built using itself. The entire codebase—the KISS Agent framework, the Sorcar agent layers, the VS Code extension, and the system prompt—was developed by KISS Sorcar operating on its own repository. This self-hosting discipline provides a continuous-integration-style stress test: if the agent introduces a bug that impairs its ability to function, we ask the agent to fix it by analyzing the trajectory and code. The simplicity of the layered architecture was both a prerequisite for and a consequence of this bootstrapping process. The simplicity of the framework reduced the number of bugs the agent introduced. The five core agent classes remain compact: the KISS Agent comprises 463 lines, the Relentless Agent 431 lines, the Sorcar Agent 685 lines, the Chat Sorcar Agent 497 lines, and the Worktree Sorcar Agent 848 lines—a total of roughly 2,924 lines of code (counting only significant lines, i.e. excluding empty lines, comment-only lines, and docstrings of private methods). Despite the Sorcar and Chat layers absorbing Docker-aware tool variants, parallel

sub-agent orchestration, persistent chat-session bookkeeping, MCP integration, slash-command dispatch, and worktree concurrency safety, the framework has remained within this compact footprint and within the layered, single-concern design.

In the project, we deliberately prioritize output quality over speed. In our experience, using a weaker or cheaper model often forces the developer to discard the agent's work and retry, ultimately increasing the total cost of completing a task. Conversely, giving a frontier model adequate time to validate its own output—running linters, type checkers, and tests before declaring success—reduces the "slop" (low-quality, subtly incorrect code) common in faster but less thorough agents. We expect token costs and inference latencies to continue to fall [Gao et al., 2025], making this quality-first posture increasingly practical. In the meantime, the code produced by our agent is well-organized, simple, and idiomatic.

We evaluate on Terminal Bench 2.0 and achieve a 62.2% overall pass rate using Claude Opus 4.6, compared with Claude Code (58%) and Cursor Composer 2 (61.7%) [Cursor Research, 2026] on the same benchmark (Section 4). These results are notable because we did not tune our prompts or any model specifically for the Terminal Bench 2.0.

In KISS Sorcar, we deliberately kept the system simple. We included only the agent technologies necessary for KISS Sorcar to function as a general-purpose software engineering assistant. We showed that a simple agent framework, without sophisticated agent technologies such as trajectory compaction and asynchronous multi-agent orchestration, was sufficient to build KISS Sorcar. By building KISS Sorcar using itself and matching or exceeding both Cursor and Claude Code, we found that established software engineering techniques and principles are important for building reliable agent systems.

Outline. Section 2 presents the five-layer agent architecture and its motivating design principles. Section 3 describes how the framework supports AI discovery, GEPA prompt optimization, and repository optimization through single-prompt drivers. Section 4 reports evaluation results on Terminal Bench 2.0. Section 5 covers the user-facing features of the VS Code extension, CLI, and web app. Section 6 details the system prompt. Section 7 illustrates painless software engineering through a real development session. Section 8 discusses related work, and Section 9 concludes.

2 Agent Architecture

We initially built the KISS Agent Framework to rapidly prototype and experiment with various prompt optimization techniques, such as Gepa [Agrawal et al., 2026], and evolutionary algorithms for algorithmic and code optimization, such as AlphaEvolve [Novikov et al., 2025] and OpenEvolve [Algorithmic Superintelligence, 2025]. We focused heavily on keeping the agent framework simple so we could rapidly prototype and experiment with ideas. The simplicity of the framework also enabled coding agents to write simple, bug-free code. We ultimately did not use any prompt optimization techniques when creating the system prompt for KISS Sorcar, as we hand-tuned it based on our long-term experience with KISS Sorcar and its behavior. The KISS Agent Framework uses five agent layers in a layered architecture combining composition and inheritance. Each layer delegates upward for the concerns it does not own.

2.1 KISS Agent

The KISS Agent is the innermost execution unit. It implements a standard ReAct loop [Yao et al., 2023b] with the following characteristics.

```python
from kiss.core.kiss_agent import KISSAgent

def calculate(expression: str) -> str:
    """Evaluate a math expression."""
    return str(eval(expression))

agent = KISSAgent(name="Math Buddy")
result = agent.run(
    model_name="gemini-2.5-flash",
    prompt_template="Calculate: {question}",
    arguments={"question": "What is 15% of 847?"},
    tools=[calculate]
```

```
)
print(result)  # 127.05
```

Listing 1: A complete KISS agent with a single tool.

Native function calling. We register tools as ordinary Python callables. The agent builds an OpenAI-compatible tool schema once at setup time and caches it, avoiding redundant schema construction on every LLM call. A special `finish` tool signals task completion and returns the result to the caller.

Step, token, and budget tracking. At every step, the agent extracts input and output token counts from the API response, computes the dollar cost using a per-model pricing table, and updates both a local budget counter and a global (cross-agent) budget counter protected by a class-level lock. The agent checks three limits before each step: the per-agent budget, the global budget, and the maximum step count.

Error resilience. The agent retries transient API errors (rate limits, server errors) up to a configurable threshold of consecutive failures. It detects non-retryable errors (authentication failures, permission denials) and raises them immediately.

Non-agentic mode. When tools are not needed, the agent can run a single generation without the ReAct loop, which is useful for summarization or question-answering sub-tasks.

Listing 1 shows a complete, working agent in under ten lines of code. The developer defines an ordinary Python function (`calculate`), instantiates a `KISSAgent`, and calls its `run` method with a model name, a prompt template, template arguments, and a list of tools. The framework automatically handles tool-schema generation, the ReAct loop, and budget tracking.

The KISS Agent is stateless across runs: each call to its run method resets the conversation, token counters, and tool registry. This makes it safe to reuse a single agent instance for multiple sequential tasks.

2.2 Relentless Agent

The Relentless Agent wraps a KISS Agent in a continuation loop. Its core contribution is the ability to execute tasks that exceed a single context window by breaking them into sub-sessions.

Rather than investing in context-compaction techniques, we adopt a simple continuation protocol: when a sub-session exhausts its context window or step budget, the agent produces a *structured summary* of every action taken so far—chronologically ordered, with explanations and relevant code snippets—and a fresh sub-session resumes from that summary. This approach is related in spirit to Reflexion [Shinn et al., 2023], which feeds verbal self-critiques back into subsequent trials, but uses a Reflexion-like technique to continue a task. While developing KISS Sorcar, we found in our experience that a naïve instruction to "summarize the current context" produced poor continuations; requiring a *step-by-step chronological account with code snippets* improved coherence across sub-sessions. A potential limitation is that summaries may grow unwieldy for multi-day tasks; in practice, we have not encountered this problem even for tasks spanning several hours, but a thorough evaluation of summary scaling remains future work.

Continuation protocol. The `finish` tool exposed to the inner KISS Agent accepts three fields: a success flag, a continue flag, and a summary. When the agent sets `is_continue=True`, the Relentless Agent starts a new sub-session with a fresh context window. The prompt for the new session includes a chronologically ordered list of all prior attempt summaries and instructs the agent not to redo completed work. The continuation prompt template is:

```
# Task Progress (Continuation {continuation_number})

{progress_text}

# Continue
- Complete the rest of the task.
- **DON'T** redo completed work.
- If you have been retrying the same approach without progress,
  step back and rethink the strategy from scratch.
```

Forced continuation on failure. If a sub-session raises an exception (e.g., the step limit is hit before the agent calls finish), the Relentless Agent does not abort. Instead, it saves the full trajectory to a temporary file, spawns a separate summarizer agent to read the trajectory and produce a concise summary, and then uses that summary as the progress text for the next sub-session. This ensures that even crashed sessions contribute useful context to subsequent attempts. The summarizer receives the following prompt:

```
# Summarizer

The trajectory of the agent is stored in the file: {trajectory_file}

# Instructions
- Read the trajectory file and analyze it. The trajectory file
  could be large.
- Return a precise chronologically-ordered list of things the
  agent did with the reason for doing that along with relevant
  code snippets
```

To force the agent to self-continue before hitting the step limit, we augment the system prompt with an instruction that fires near the end of the budget:

```
# MOST IMPORTANT INSTRUCTIONS
- **At step {step_threshold}: you MUST call
  finish(success=False, is_continue=True,
  summary="precise chronologically-ordered list of things
  the agent did with the reason for doing that along with
  relevant code snippets")** or if the task is not complete
  and you are at risk of running out of steps or context
  length.
- Work dir: {work_dir}
- Current process PID: {current_pid} -- NEVER kill this
  process.
```

2.3 Sorcar Agent

The Sorcar Agent adds the tools that make the system useful for software development and general-purpose automation.

Coding tools. We provide four core tools: a shell command executor with streaming output, a file reader, a precise string-based file editor, and a file writer. The shell executor supports a configurable timeout, streams output to the user interface in real time, and respects a stop event that allows the user to cancel a running command. Note that we kept the tool names (Bash, Edit, Write, Read) the same as in Claude Code because the underlying Anthropic models do not make mistakes with these tool names.

Browser automation. A web-use tool provides programmatic browser control: navigating to URLs, reading page accessibility trees, clicking elements, typing text, pressing keys, scrolling, and taking screenshots. This enables the agent to research documentation, verify deployed applications, and interact with web-based tools. It uses the open-source Chromium browser using the Playwright library.

Parallel sub-agents. A parallel execution tool spawns independent Sorcar Agent instances in a thread pool. Each sub-agent gets its own LLM context and tool set. This is useful for embarrassingly parallel tasks such as summarizing multiple files or researching independent topics. We collect the results and return them in input order.

User interaction. An ask-user-question tool allows the agent to pause execution and request clarification from the user. In the VS Code integration, this renders as a text input in the sidebar; in CLI mode, it reads from standard input.

Docker isolation. When a Docker image is specified, we replace the coding tools with Docker-aware variants that execute commands inside a container, providing an additional layer of sandboxing for untrusted tasks.

Dynamic model switching with `set_model`. A `set_model` tool, registered automatically on every Sorcar Agent, lets the agent hand the live conversation off to a different LLM at any point during a task

without restarting it. The tool accepts a model name (for example `"gpt-5.5"`, `"claude-opus-4-7"`, or `"gemini-2.5-flash"`), constructs the new backend through the same factory that built the original model, copies the conversation history and the cumulative usage counters into the new backend, rebuilds the cached tools schema in the new provider's dialect (Anthropic and OpenAI schemas differ in subtle ways), persists the choice to disk so that the next task in the same chat session reuses it, and returns a confirmation string back to the agent loop. If no model has been instantiated yet (deferred-startup case), it simply updates the agent's default `model_name` for the next run; if the requested model is already active, it is a no-op.

This single tool is what enables *multi-model, multi-vendor workflows inside a single task*. Three idiomatic patterns recur in our usage: (i) a *scout-then-edit* pattern where a cheap fast model (e.g. `gemini-2.5-flash`) reads the repository, grep-searches for relevant call sites, and gathers context, then `set_model("claude-opus-4-7")` hands off to a stronger reasoner for the actual edit; (ii) a *generate-then-review* pattern where a primary model produces a change and the agent calls `set_model("gpt-5.5")` to switch to a second-opinion model that re-reads the diff, runs the tests, and verifies the change against the original request; and (iii) a *cost-aware long-loop* pattern where an open-ended optimization or discovery task (Section 3) spends a cheap model on bookkeeping and journal-keeping but pays for a frontier model only at the decision points. Because the conversation transcript and usage counters are carried across the switch, all subsequent budget accounting and continuation summaries remain coherent regardless of how many times the model is changed.

The following three prompts, reproduced from `src/kiss/INJECTIONS.md`, illustrate what this tool layer empowers. They are not API calls or special modes: each is a single natural-language instruction that can be pasted into the IDE or CLI. Their power comes from the combination of dynamic model switching, coding tools, test execution, parallel sub-agents, and later layers' persistence and worktree isolation.

Primary model plus independent review.

```
Use claude-opus-4-7 model for all tasks including coding, bug
fixing, and test creation. Use gpt-5.5 model (not codex) for
thorough review and debugging of the work done by the other
model. Check if the other model has missed some code or has
introduced bugs. No need to check if the models exist.
```

Self-review, reproduce, fix, and repeat.

```
Can you review the updates made in the last task using gpt-5.5
(non codex) and find bugs or missing code. Then write end-to-end
tests reproducing the bugs reported by the review, fix them, and
test them using claude-opus-4-7? Run all the tests in parallel
and fix bugs after thoroughly reviewing the fixes with gpt-5.5
(non codex). Repeat the process until you fail to reproduce the
bugs reported by the review done by gpt-5.5 (non codex). No need
to check if the models exist.
```

Three-model generation and review.

```
Use openrouter/z-ai/glm-5.2 model for all tasks including coding,
bug fixing, and test creation. ALWAYS use gpt-5.5 model (not
codex) to carefully and thoroughly review and debug the work done
by openrouter/z-ai/glm-5.2 for bugs and missing code. Then use
claude-opus-4-7 to do the same thing.
```

These prompts show that `set_model` turns model choice from a session-level setting into a programmable task-level resource. A user can ask one model to implement and test, another model to review the result, and a third model to audit both the implementation and the review without leaving the same conversation. Combined with the Sorcar Agent's real coding tools and parallel test execution, this enables inexpensive models to do routine work while stronger or independent models are reserved for the high-value verification steps where missed code, subtle bugs, and inadequate tests are most likely to matter.

2.4 Chat Sorcar Agent

The Chat Sorcar Agent adds multi-turn conversation persistence.

Chat sessions. We assign each task to a chat session identified by a stable chat ID. The agent persists tasks and their results to a local database (`sorcar.db`). When a new task arrives within the same

chat session, the agent loads prior tasks and results and prepends them to the prompt as numbered context entries, allowing the LLM to reference earlier work.

Bounded chat context. To prevent unbounded growth of the prompt as a chat session accumulates many tasks, the agent caps the number of in-context history entries at `MAX_TASKS=10`. When the cap is exceeded, it preserves the first two entries (which typically establish the user's overall intent for the session) and the most recent entries, dropping the middle entries that are least likely to be referenced. This keeps the context relevant and bounded while retaining both the session's framing and its current state.

Session management. The agent supports three operations: starting a new chat (with a fresh chat ID), resuming a chat by task description (which looks up the corresponding chat ID), and resuming by explicit chat ID. This enables both automatic session continuity in an IDE and manual session selection from the command line.

Frequent task tracking. Each time a task is executed, the agent records the task description in a frequency table. The IDE sidebar surfaces the most frequent tasks so users can re-issue them with one click, turning recurring requests ("run the test suite," "regenerate the changelog") into a click-to-replay experience.

Metadata persistence. After each task, the agent records metadata including the model used, working directory, software version, token count, cost, and whether the task used parallel execution or worktree isolation. This audit trail supports cost analysis and debugging.

2.5 Worktree Sorcar Agent

The Worktree Sorcar Agent is the outermost layer and the one that users interact with in the VS Code extension and the Sorcar web app. Its defining feature is git-worktree isolation.

Branch-per-task. When a task starts, the agent creates a new git branch and a corresponding worktree directory. The branch name encodes the chat ID and a timestamp for uniqueness. All agent modifications happen inside the worktree; the user's main working tree remains untouched.

Dirty-state preservation. If the user's main working tree has uncommitted changes, the agent copies them into the worktree and creates a baseline commit. This ensures the agent sees the same state as the user, while keeping the user's actual index and working tree clean. During merge, we use cherry-pick from the baseline commit to replay only the agent's changes, excluding the dirty-state snapshot.

Concurrency safety. A per-repository file lock serializes the checkout, stash, merge, and pop sequence so that concurrent tabs in the IDE cannot interleave operations on the same repository. Thread-local storage isolates per-task state (stream parsing buffers, bash output buffers, recording state) so that stopping one task does not corrupt another.

Crash recovery. We store all worktree state in git itself (branch names, git config entries) rather than in sidecar files. On process restart, the agent queries git for any pending branch matching its chat ID prefix and reconstructs all instance attributes from git config, enabling recovery.

Graceful fallback. If the working directory is not inside a git repository, if the repository has no commits, or if HEAD is detached, the agent falls back to direct execution without worktree isolation, ensuring it never fails due to git preconditions.

3 AI Discovery, GEPA Optimization, and Repository Optimization

Beyond per-task coding assistance, KISS Sorcar can be driven as an autonomous *research and optimization driver*: the user issues a single high-level objective with explicit numeric stopping conditions (a target accuracy, a target latency, a budget cap, a "do not stop until…" clause), and the agent then runs an open-ended exploration loop—spawning experiments, reading logs, mutating its own prompts or code, journalling what worked and what failed—until the targets are met or the budget is exhausted. The capabilities described in Section 2 were not designed for this use case individually, but together they cover exactly what an open-ended driver needs: the Relentless Agent's structured-summary continuation (Section 2.2) lets multi-hour or multi-day loops survive context exhaustion; the Sorcar Agent's parallel sub-agents and streamable shell executor (Section 2.3)

parallelize the inner experiments and monitor long-running commands in real time; the `set_model` tool lets a cheap model do the routine bookkeeping while a stronger model is engaged only at decision points; the Chat Sorcar Agent persists every tool call to `sorcar.db`, providing a replayable ground truth for self-reflection (Section 2.4); and the Worktree Sorcar Agent's branch-per-task isolation (Section 2.5) keeps every trial reproducible and rollback-friendly.

We ship multiple sample task templates in `src/kiss/SAMPLE_TASKS.md` that exercise this pattern. Each is a single natural-language prompt that fits in a chat box; the agent does the rest.

3.1 AI Discovery

AI-driven discovery casts scientific and algorithmic problem-solving as an evolutionary search over programs: an outer loop maintains a population of candidate solutions (each a complete piece of source code, a prompt, or a model recipe), an LLM proposes new candidates by mutating or recombining selected parents, and an automated evaluator scores every offspring on a task-specific fitness function, with the fittest individuals fed back as parents for the next generation [Romera-Paredes et al., 2024, Novikov et al., 2025, Algorithmic Superintelligence, 2025, Lange et al., 2025, Cheng et al., 2025, Agrawal et al., 2026]. This is exactly the structure of a classical genetic algorithm—selection, mutation, crossover, replacement—except that the variation operators are realized by a large language model conditioned on natural-language critiques of past failures rather than by random bit-flips, and the genotype is human-readable source code rather than a fixed-length bitstring. DeepMind's *FunSearch* [Romera-Paredes et al., 2024] pioneered this paradigm by pairing a code-LLM with an island-based evolutionary population and using execution feedback as fitness, discovering new constructions for the cap-set problem and improved bin-packing heuristics; *AlphaEvolve* [Novikov et al., 2025] scaled the same recipe to full programs across mathematics, hardware design, and Google infrastructure; *OpenEvolve* [Algorithmic Superintelligence, 2025] added MAP-Elites quality-diversity and an artifact side-channel to the open-source community; *ShinkaEvolve* [Lange et al., 2025] sharpened sample efficiency with novelty-aware parent sampling and an LLM-ensemble bandit; *ADRS* [Cheng et al., 2025] demonstrated that this evolutionary loop transfers cleanly to systems-performance research whenever the evaluator is a runnable workload; and *GEPA* [Agrawal et al., 2026] showed that the same selection–mutation–crossover skeleton optimizes natural-language prompts when fitness is measured by trajectory-level reflection rather than scalar reward. Common to all of these is a generic contract—natural-language objective, code or prompt as the unit of mutation, automated evaluator as the verifier, journal of tried ideas, and explicit anti-reward-hacking and generalization clauses—and KISS Sorcar's AI Discovery template inherits exactly this contract while supplying every primitive needed to instantiate it (budget-aware tool use, Relentless continuation, parallel sub-agents, branch-per-trial worktree isolation, replayable trajectories in `sorcar.db`, and dynamic `set_model` switching).

The exact prompt shipped in `src/kiss/SAMPLE_TASKS.md` is reproduced verbatim below; the user fills in only the data path and runs it as a single chat message:

> *Sorcar for AI Discovery: Can you discover the lightest and fastest AI model that will give the best accuracy and recall on the data at «/path/to/data» at the cheapest price? Analyze the data and search the internet extensively to propose the first few models. Implement and experiment with each of your proposals. Note down the ideas you used to optimize the accuracy/recall and speed/cost metrics achieved in a file, so that you can use the file to not repeat ideas that have already been tried and/or failed. You can also use the file to combine ideas that have been successful in the past. Separate 20% of the data for evals, and your discovery strategy must not look at the evals data. Use `'lambda'` CLI to train your models on GPUs and evaluate if needed. Total budget for Lambda Labs is $1000. Experiment with a smaller subset of data and fewer parameters in a model to do experiments quickly, and then extrapolate. Use internet search extensively at every step. MAKE SURE THAT YOU DO NOT DO REWARD HACKING OR CHEATING IN THE MODELS OR AGENTS YOU ARE IMPLEMENTING TO FIT DATA. YOUR SOLUTION MUST GENERALIZE BEYOND THE DATA PROVIDED. Do not STOP until accuracy/recall reaches 95% on evals and you can process each query in less than 600 seconds and under 50 USD per query amortized over all queries. Create an html report with diagrams and illustrations in `./reports` and open it in the user's default browser?*

Advantages of the prompt-as-driver formulation. Casting AI discovery as a single natural-language prompt rather than as a bespoke evolutionary harness has several practical advantages.

(i) Zero infrastructure. Unlike FunSearch, AlphaEvolve, OpenEvolve, and ShinkaEvolve, the user does not configure an island topology, write an evaluator plug-in, or stand up a programs database; the agent's persistence layer (`sorcar.db`) and worktree isolation provide the same services for free. *(ii) Explicit numeric stopping rule.* The clause "Do not STOP until accuracy/recall reaches 95% ... and ... under 50 USD per query" converts the open-ended search into a verifiable contract: the Relentless Agent's continuation protocol keeps the loop alive across context windows precisely until those numbers are reached or the $1000 Lambda Labs budget is exhausted, with no human in the inner loop. *(iii) Anti-cheating clauses in the prompt itself.* The capitalized *"MAKE SURE THAT YOU DO NOT DO REWARD HACKING OR CHEATING... YOUR SOLUTION MUST GENERALIZE BEYOND THE DATA PROVIDED"* directive, together with the mandatory 20% held-out eval split that "the discovery strategy must not look at," encodes the same generalization invariants that ADRS and ShinkaEvolve enforce through their evaluators—but without writing a single line of harness code. *(iv) Journal-driven memory.* The instruction to "note down ideas ... in a file" turns the file system into a long-term programs database analogous to the FunSearch programs database, but inspectable and editable by the user; combined with branch-per-trial worktree isolation, every candidate is reproducible and rollback-friendly. *(v) Cheap exploration, expensive verification.* "Experiment with a smaller subset of data and fewer parameters in a model to do experiments quickly, and then extrapolate" pushes the agent toward AlphaEvolve-style cascade evaluation: many cheap rollouts on a sub-sample, a few expensive full-scale evaluations. Combined with `set_model`, the agent can additionally switch to a cheap LLM for routine bookkeeping and reserve a frontier model for the harder algorithmic decisions and the end-of-step review pass. *(vi) Auditable output.* The mandatory HTML report in `./reports/`, opened in the user's default browser, makes every multi-hour run reviewable at a glance instead of buried in chat history. *(vii) Parallel candidate evaluation.* The Sorcar Agent's parallel sub-agents evaluate independent candidate models on disjoint slices of the data in parallel, mirroring the population-level parallelism of evolutionary frameworks without any additional orchestration code.

3.2 GEPA Prompt Optimization

Prompt optimization is the problem of automatically rewriting one or more natural-language prompts inside a (possibly compound) LLM system so that a downstream task metric is maximized, without access to module-level labels or weight gradients [Yang et al., 2024a, Pryzant et al., 2023, Fernando et al., 2023, Opsahl-Ong et al., 2024, Yuksekgonul et al., 2024, Khattab et al., 2024, Agrawal et al., 2026]. The shared template is an outer loop that proposes candidate prompts, evaluates each on a labeled minibatch, and keeps the best for the next round—a *discrete* analogue of stochastic optimization in which the LLM itself serves as the variation operator. OPRO [Yang et al., 2024a] formalized the loop by passing the LLM a history of past prompts together with their numeric scores and asking for a better one; ProTeGi/APO [Pryzant et al., 2023] replaced scalar rewards with natural-language "gradients"—LLM-written critiques of the current prompt—and propagated them through beam search; Promptbreeder [Fernando et al., 2023] evolved a population of task-prompts *and* the mutation prompts that mutate them, in a self-referential genetic algorithm; MIPROv2 [Opsahl-Ong et al., 2024] extended these ideas to multi-stage LM programs in DSPy [Khattab et al., 2024] by jointly searching over instructions and few-shot demonstrations under a stochastic minibatch surrogate; and TextGrad [Yuksekgonul et al., 2024] cast the whole compound system as an autograd-style graph through which textual feedback is backpropagated. GEPA [Agrawal et al., 2026] unifies these threads in a single "Genetic-Pareto" optimizer that (i) samples full trajectories of a system—tool calls, intermediate model responses, and outputs—rather than only final answers, (ii) uses reflective natural-language critiques as mutation operators in the spirit of ProTeGi and TextGrad, (iii) maintains a Pareto frontier of complementary prompts in the spirit of MAP-Elites quality-diversity, and (iv) combines complementary lessons across frontier nodes by crossover. GEPA reports up to 20% absolute gain over GRPO with 35$\times$ fewer rollouts on six tasks and beats MIPROv2 by over 10%, establishing reflective prompt evolution as a sample-efficient alternative to reinforcement-learning fine-tuning. The Sorcar GEPA template instantiates exactly this algorithm against a `ChatSorcarAgent` target, with the agent's per-tool-call trajectory store (`sorcar.db`, Section 2.4) supplying the trajectory data on which reflection operates.

The exact prompt shipped in `src/kiss/SAMPLE_TASKS.md` is reproduced verbatim below; the user fills in only the data path and runs it as a single chat message:

Sorcar GEPA Prompt Optimizer: Can you optimize a prompt for a ChatSorcarAgent of the kiss-agent-framework Python library using the following GEPA algorithm on the data at «url_or_db_file_of_data» using claude-opus-4-7? You can find the trajectory events of an agent execution in `~/.kiss/sorcar.db` after the agent has finished its execution. Split the dataset into 50% dev set and 50% val set.

RUN_GEPA: Sample 100 data points from the val set and call it sval set. Maintain a pareto frontier in the folder `./pareto` where we have a sub-folder for each node in the frontier. A node contains a prompt file (`prompt.md`) and a json file, say `score.json`, containing the list of datapoints (ids) from the sval set that were correctly predicted with the prompt. When you add a node to the pareto frontier make sure that the list of correctly predicted datapoints is not a subset or equal to an existing list of datapoints in some node in the frontier. If such a node exists, do not add the new node. After adding a node, remove all nodes whose list of datapoints is a subset or equal to the list of datapoints in the added node. Then run the following algorithm.

1. *pick a node from the pareto frontier with probability 0.5*
   (a) *sample a minibatch of 5 datapoints from the dev set*
   (b) *run the agent with the prompt from the node on the minibatch*
   (c) *if the agent incorrectly predicts for some datapoints, analyze and reflect on the trajectory events of the agent on those datapoints available at `~/.kiss/sorcar.db` and propose a new prompt which will fix the mistakes made by the agent on datapoints incorrectly predicted.*
   (d) *if the agent predicts correctly on the minibatch, then evaluate it on the sval set and create the list of datapoints on which the agent with the new prompt predicts correctly.*
   (e) *Add the new prompt and the list of datapoints to the pareto frontier*
2. *pick two nodes from the pareto frontier randomly with the remaining probability.*
   (a) *sample a minibatch of 5 datapoints from the dev set*
   (b) *merge the prompts from the two nodes into a new prompt.*
   (c) *if the agent predicts correctly on the minibatch with the new prompt, then evaluate it on the sval set and create the list of datapoints on which the agent with the new prompt predicts correctly.*
   (d) *Add the new prompt and the list of datapoints to the pareto frontier*
3. *Repeat steps 1 and 2 until there is no change in the prompt after 3 iterations.*

END_RUN_GEPA. Repeat RUN_GEPA until there is no change in the prompt after 3 iterations.

In each step, keep track of the best prompt which has the maximum number of successfully predicted datapoints in `./pareto/optimal.md`. MAKE SURE THAT YOU DO NOT DO REWARD HACKING OR CHEATING IN THE AGENT YOU ARE IMPLEMENTING TO FIT DATA. YOUR SOLUTION MUST GENERALIZE BEYOND THE DATA PROVIDED. Use internet search extensively at every step. Do not worry about budget. Create an html report with diagrams and illustrations in `./reports` and open it in the user's default browser. Do NOT STOP until you could not improve the accuracy and recall after three consecutive rollouts. Use `gpt-5.5` model (not codex) for thorough review of the work done at every step by the other model.

Advantages of the prompt-as-driver formulation. Expressing GEPA as a single chat prompt rather than as a bespoke Python library has several concrete advantages for the user. *(i) No optimizer code.* Unlike OPRO, MIPROv2, ProTeGi, TextGrad, or the reference GEPA implementation, the user does not write or maintain a Pareto-frontier data structure, a minibatch sampler, or a reflection-prompt template; the agent constructs the frontier as a flat `./pareto/` directory tree (one folder per node containing `prompt.md` and `score.json`) that is trivially inspectable and version-controllable by the user. *(ii) Persistence layer as the trajectory store.* Because every tool call, model response, and cost emitted by the target `ChatSorcarAgent` is already written to `~/.kiss/sorcar.db` by the metadata-persistence path of the Chat Sorcar Agent (Section 2.4), the GEPA reflector can re-read any past trajectory at the exact tool call that produced the failure rather than guessing from a final answer alone—realizing the trajectory-level reflection that makes GEPA outperform scalar-reward RL by 6–20% in the original paper, but at zero integration cost. *(iii) Two-model cascade through `set_model`.* The prompt explicitly designates a primary model (e.g. `claude-opus-4-7`) for the bulk of the rollouts and an independent reviewer model from a different vendor (e.g. `gpt-5.5`, "not codex") only for the reflection, prompt-rewrite, and end-of-step review steps, mirroring the

two-model setup used by GEPA and AlphaEvolve while spending the second model's tokens only where they pay off. *(iv) Independent review.* The closing clause "*Use `gpt-5.5` model (not codex) for thorough review of the work done at every step by the other model*" makes a different model audit the rewriter's proposals, catching reward-hacking edits that a single-model loop would silently accept. *(v) Explicit termination contract.* "*Do NOT STOP until you could not improve the accuracy and recall after three consecutive rollouts*" converts the open-ended evolution into a verifiable stopping rule that the Relentless Agent's continuation protocol (Section 2.2) enforces across context windows. *(vi) Anti-cheating and held-out evaluation in the prompt.* The capitalized *"MAKE SURE THAT YOU DO NOT DO REWARD HACKING OR CHEATING... YOUR SOLUTION MUST GENERALIZE BEYOND THE DATA PROVIDED"* clause, together with the 50/50 dev/val split and the disjoint 100-example `sval` sub-val set, encodes the same generalization invariants that the GEPA paper and ShinkaEvolve enforce through their evaluators—but without writing a single line of harness code. *(vii) Auditable output.* The mandatory HTML report in `./reports/`, opened in the user's default browser, makes the evolution trace and the Pareto-frontier history reviewable at a glance.

3.3 Repository Optimization

Repository optimization is the problem of taking an existing codebase together with a runnable command—a build, a benchmark, a server, an inference script, or a training loop—and improving its observable metrics (speed, accuracy, recall, cost) by repeatedly editing the source, re-running the command, reading the output, and proposing the next edit [Yang et al., 2024b, Xia et al., 2024, Jimenez et al., 2024, Ouyang et al., 2025, Wang et al., 2024b, Gauthier, 2023, Anthropic, 2025b, Cheng et al., 2025]. This is structurally the same outer loop as AI discovery, but with two distinguishing features. First, the genotype is not a single self-contained file but a *whole repository*, so the agent must reason about cross-file dependencies, build artifacts, and dynamic runtime behavior rather than a fixed harness; this is the regime in which SWE-agent [Yang et al., 2024b] showed that a purpose-built agent-computer interface (the ACI) outperforms naïve prompt-only baselines on SWE-bench, in which Agentless [Xia et al., 2024] showed that a fixed localize–repair–validate pipeline can rival agentic ones on SWE-bench Lite, and in which OpenHands [Wang et al., 2024b], Aider [Gauthier, 2023], and Claude Code [Anthropic, 2025b] ship repository-level coding interfaces as production tools. Second, the fitness function is a real running process—e.g. wall-clock latency, throughput, GPU memory, accuracy on a validation set, or cloud-dollar cost—so the agent must launch the command in the background, monitor its streaming output in real time, and abort speculative runs the moment a partial measurement renders them moot; KernelBench [Ouyang et al., 2025] shows that even frontier reasoning models initially beat the PyTorch baseline on fewer than 20% of GPU-kernel optimization workloads but improve substantially once execution and profiler feedback is folded back into the prompt, and the ADRS thesis [Cheng et al., 2025] argues that exactly this profile-edit-rerun loop is where LLM-driven research is currently most productive. The Sorcar Repository Optimization template instantiates this loop as a single chat prompt, with the Sorcar Agent's streaming shell executor (Section 2.3) providing the real-time monitoring channel and the Worktree Sorcar Agent's branch-per-task isolation (Section 2.5) providing reproducible rollback of every speculative edit.

The exact prompt shipped in `src/kiss/SAMPLE_TASKS.md` is reproduced verbatim below; the user fills in the command, the target folder or URL, the metrics of interest, and the concrete numeric targets:

> *Sorcar for Optimization: Can you run the command «command» in the background and monitor its output in real time to optimize the code at «folder_name_or_url» with respect to the following metrics: «speed,accuracy,recall,cost». You can add diagnostic code which will print the metrics, such as running time at a finer level of granularity. Check for opportunities to optimize the code on the basis of the metrics information. If you discover any opportunities to optimize the metric based on the code, logs, events, and the command output, optimize the code and run the command again. Note down the ideas you used to optimize the code and the metric you achieved in a file, so that you can use the file to not repeat ideas that have already been tried and failed. You can also use the file to combine ideas that have been successful in the past. Repeat the process. Do not forget to remove the diagnostic code after the optimization is complete. You MUST NOT STOP until the metrics achieve the following values: «give_concrete_values_for_metrics». Use the internet extensively to get new ideas for optimization. Create an html report with diagrams and illustrations in `./reports` and open it in the user's default browser?*

Advantages of the prompt-as-driver formulation. Casting repository optimization as a single chat prompt rather than as a bespoke driver script has several concrete advantages. *(i) Real-time, cancellable measurement.* The Sorcar Agent's streaming shell executor (Section 2.3) lets the agent launch the user's command in the background, watch its output as it arrives, and cancel the run the moment a new edit invalidates it—without this, every speculative optimization would have to wait for a full benchmark to finish, which is exactly the bottleneck KernelBench [Ouyang et al., 2025] identifies for GPU-kernel optimization. *(ii) Branch-per-trial isolation.* The Worktree Sorcar Agent's branch-per-task isolation (Section 2.5) means each candidate optimization lands on its own `git` branch, so a regression discovered three iterations later can be reverted with a single `git` operation while preserving progress on parallel, unrelated branches—bringing the rollback discipline that AlphaEvolve and OpenEvolve achieve through a programs database into a standard developer workflow. *(iii) Cheap profile-and-edit, expensive rewrite.* Combined with `set_model`, the agent can profile and apply local edits on a cost-efficient model and switch to a stronger model only for the harder algorithmic rewrites and the final review pass, mirroring AlphaEvolve's cheap-rollout / expensive-verification cascade but inside a single chat session. *(iv) Self-instrumenting then self-cleaning.* The clause "*You can add diagnostic code which will print the metrics... Do not forget to remove the diagnostic code after the optimization is complete*" lets the agent insert finer-grained profiling exactly where its current hypothesis demands, and the system prompt's Pre-Finish Verification rules (Section 6) force the diagnostic code to be removed before the final commit so the optimized repository ships clean. *(v) Explicit numeric stopping rule.* "*You MUST NOT STOP until the metrics achieve the following values: «give_concrete_values_for_metrics»*" converts the open-ended search into a verifiable contract that the Relentless Agent's continuation protocol (Section 2.2) keeps alive across context windows until the targets are met. *(vi) Journal-driven memory.* The instruction to "*note down the ideas you used to optimize the code and the metric you achieved in a file*" turns the file system into a long-term programs database analogous to FunSearch's and AlphaEvolve's programs databases, but inspectable and editable by the user; combined with branch-per-trial isolation, every candidate is reproducible and rollback-friendly. *(vii) Internet-grounded search for ideas.* "*Use the internet extensively to get new ideas for optimization*" couples the inner-loop measurement signal to the web-research protocol of the system prompt (Section 6), so the agent can discover algorithmic improvements (cache-blocked layouts, fused operators, alternative serialization formats, faster linear-algebra libraries) rather than only local micro-optimizations. *(viii) Auditable output.* The mandatory HTML report in `./reports/`, opened in the user's default browser, makes every multi-hour optimization run reviewable at a glance, with diagrams showing the trajectory of each metric over time.

Common pattern

All three templates share the same skeleton: a high-level natural-language objective, explicit numeric stopping conditions, a journal file of tried ideas, parallel exploration where possible, branch-per-trial isolation, an anti-reward-hacking clause, an explicit instruction to use internet search extensively at every step, and a final HTML report written into `./reports/` and opened in the user's default browser. The agent's system prompt (Section 6) already encodes the engineering disciplines—read-before-modify, lint-and-test before `finish`, no fabricated source counts, no shortcuts—so the user need only state the objective, the targets, and any external constraint (such as a GPU CLI to use). In our experience, this open-ended-driver use case is where the layered architecture pays off most: each layer's narrow concern—budget, continuation, parallelism, persistence, isolation, dynamic model selection—is exactly what a long-running self-directed exploration requires, and the simplicity of the framework leaves no room for the driver to silently corrupt its own state over a multi-hour run.

4 Evaluation on Terminal Bench 2.0

Before we discuss the system prompt in a lengthy section, we describe the evaluation outcome of KISS Sorcar on the Terminal Bench 2.0, which was also recently used by the Cursor agent of Composer 2.0.

We evaluate our system on Terminal Bench 2.0,[1] a benchmark comprising 89 diverse terminal-based programming tasks, ranging from building legacy compilers and configuring servers to solving

[1] `https://www.tbench.ai/`

cryptanalysis challenges and training machine-learning models. Each task runs in an isolated Docker container; a separate verifier automatically judges the result. We use the Harbor[2] framework to orchestrate execution, and Claude Opus 4.6 as the underlying LLM. We do not modify the general system prompt or inject Terminal Bench 2.0-specific instructions during the evaluation. We carried out our evaluation on a 2025 MacBook Air 15" with an M4 processor and 24GB RAM.

4.1 Setup

We run 5 independent trials per task. The agent is `SorcarHarborAgent`, a thin Harbor adapter that installs and invokes the Sorcar CLI inside each container. We hard-skip 9 tasks that we verified to be infeasible for Opus 4.6 across 6+ prior attempts (e.g. CompCert compilation, Windows 3.11 GUI installation, video OCR) to save time and token cost. Skipped tasks still count as failures.

4.2 Aggregate Results

Table 2 summarizes the aggregate statistics.

Table 2: Terminal Bench 2.0 aggregate results (89 tasks, 5 trials each, Claude Opus 4.6).

Metric	**Value**
Total tasks	89
Overall pass rate	62.2% (277/445)
pass@any (at least 1/5 passes)	78.7% (70/89)
pass@all (all 5 pass)	43.8% (39/89)
Always-fail tasks	19
Always-pass tasks	39
Mixed-result tasks	31
Median cost per trial	$0.45
Mean cost per trial	$0.90
Median duration per trial	202 s
Mean duration per trial	446 s

The 62.2% overall pass rate is comparable to other agents using the same underlying model: at the time of writing, Claude Code (also Opus 4.6) scores approximately 58% on the Terminal Bench 2.0 leaderboard, and Cursor's Composer 2—a custom fine-tuned model trained with large-scale reinforcement learning [Cursor Research, 2026]—achieves 61.7%. This suggests that the layered architecture and the structured system prompt described in Sections 2 and 6 contribute beyond what the base model alone provides.

4.3 Task-Level Breakdown

Consistently solved tasks (39 of 89). These include cryptanalysis (FEAL differential), game-playing (chess best move), git operations (leak recovery), server configuration (gRPC key-value store, PyPI server, NGINX logging), data processing (resharding), formal verification (Coq `plus_comm`), ML inference (HuggingFace model serving, LLM batching scheduler), and system emulation (QEMU startup). These tasks span systems, security, data engineering, and formal methods.

Consistently failed tasks (19 of 89). The failures cluster into three categories: (1) tasks requiring graphical or multimedia capabilities unavailable in the container (video processing, Windows 3.11 GUI, MTEB leaderboard scraping, extracting moves from video), (2) tasks demanding very long or resource-intensive builds that exceed the container's time or memory limits (CompCert, Doom for MIPS, Caffe CIFAR-10, training fastText on Yelp data), and (3) tasks with niche domain-specific requirements that the model struggles to satisfy (DNA insertion, OCaml GC patching, polyglot C/Python binaries, protein assembly, cell segmentation).

Mixed-result tasks (31 of 89). Tasks such as `write-compressor` (3/5), `crack-7z-hash` (4/5), and `feal-linear-cryptanalysis` (4/5) succeed in most trials but occasionally fail due to non-determinism in the model's reasoning or timing-sensitive environment interactions. Conversely,

[2] `https://github.com/harbor-framework/harbor`

`cancel-async-tasks` (1/5) and `dna-assembly` (1/5) succeed rarely, suggesting they are at the boundary of the model's capability.

Leaderboard context. KISS Sorcar does not score as high as other coding agents reported at the Terminal Bench 2.0 leaderboard, but these results are notable because we did not tune our prompts or any model specifically for the Terminal Bench 2.0. We used the general system prompt and Claude Opus 4.6 without modification. Regarding the lower score compared to other coding agents, recent analysis has found widespread cheating on popular agent benchmarks, including Terminal Bench 2.0: the top three submissions commit harness-level cheating (e.g. leaking verifier code or answer keys into the agent's environment), and task-level cheating (e.g. Googling answers, mining git history, hardcoding test outputs) affects 28+ submissions across 9 benchmarks [Stein et al., 2026a,b]. Separately, we discovered, using an automated benchmark audit agent, that 45 confirmed hacking solutions across 13 widely used benchmarks exhibited process-isolation failures, answer leakage, and weak test assertions that allow perfect scores without solving a single problem [Wang et al., 2026].

5 User-Facing Features

We release our system as three coordinated surfaces over a single local daemon: a VS Code extension, a Claude-Code-style command-line interface (`sorcar`), and a browser/mobile web app. While the underlying agent architecture (Sections 2 and 6) already differs from existing AI coding assistants, the user-facing design introduces several features that differ from those in existing IDE assistants such as GitHub Copilot [GitHub, 2021], Cursor [Cursor, 2024], Windsurf [Codeium, 2024], Devin [Cognition Labs, 2024], and Aider [Gauthier, 2023]. We describe these features below.

5.1 Multi-Model, Multi-Vendor Workflows

KISS Sorcar ships with a catalog of 504 models spanning nine provider categories—68 OpenAI, 13 Anthropic, 20 Gemini, 84 Together AI, 8 Z.AI (GLM family), 6 Moonshot AI (Kimi/Moonshot), 295 OpenRouter, 3 Claude Code CLI (exposed under the `cc/*` namespace), and 7 OpenAI Codex CLI (`codex/*` namespace) entries—of which 488 are generation-capable, 329 are function-calling-capable, and 7 are embedding models. A task may mix models simply by issuing prompts that reference a different model id; the framework swaps the backend per call without reinitializing the agent or the chat session. Beyond bundled providers, `-endpoint/-header` (and the corresponding `update_settings` controls inside the agent) configure any OpenAI-compatible HTTP server—including local models served by, for example, vLLM or llama.cpp—which keeps the bring-your-own-key, local-first posture even for self-hosted backends. Because keys and prompts travel directly from the developer's machine to the chosen provider, no Sorcar-operated intermediary observes the traffic.

5.2 Dynamic Steering of a Running Task

A common failure mode of long-running AI tasks is that the user notices a misunderstanding, an additional constraint, or a wrong direction *while* the agent is already executing. KISS Sorcar lets the user type a follow-up natural-language message at any moment during a live task, and that message is appended to the running conversation before the next model step—without halting the in-flight tool call, discarding the work done so far, or starting a new task from scratch.

Steering is uniform across all three surfaces. In the VS Code sidebar, the running-task input bar accepts follow-up messages instead of starting a new task. In the `sorcar` CLI, a bordered input box is pinned to the bottom of the terminal while agent output keeps scrolling above it, so the user can compose a message at any time without interrupting the stream. The browser and mobile surfaces expose the same input through the local daemon. In every case the new message is delivered to the running agent before its next model turn and enters the live conversation as a user message, so the model sees the new guidance the moment it begins its next step.

Dynamic steering is orthogonal to the continuation mechanism of Section 2.2 (which preserves progress across context-window and step-budget boundaries) and to worktree isolation (Section 2.5); a steered task can still be committed-and-merged or discarded as a whole branch. Because external backends such as Claude Code CLI and the Codex CLI are exposed as model providers and third-party agents are exposed through the same tool layer (Section 5.6), the same user message can steer

a local KISS Agent, a worktree-isolated Sorcar task, parallel sub-agents, or work delegated to a supported external backend. We compare KISS Sorcar's dynamic steering with related in-flight control mechanisms—Claude Code Remote Control, GitHub Copilot cloud agent follow-ups, Cursor Cloud Agents, Codex queue-versus-steer mode, Aider `AI!` comments, and the LangChain/LangGraph human-in-the-loop middleware—in Section 8.1.

5.3 Real-Time Budget Accountability

AI coding assistants typically operate on a subscription model (Copilot, Cursor) or a per-seat pricing model (Devin, Windsurf), both of which obscure the per-task cost. The developer has no visibility into how many tokens a task consumed or how much it cost.

Our extension displays real-time cost tracking in the sidebar: input tokens, output tokens, cache hits, dollar cost, and elapsed time are updated at every agent step. We enforce both per-task and global budget ceilings. If a task exceeds its budget, the agent raises a hard error rather than silently accumulating charges. This transparency allows developers to make informed decisions about which tasks to delegate to the AI and how to structure prompts for cost efficiency. The KISS Agent also appends the current usage in the context so that the model is fully aware of its limits.

5.4 Integrated Browser Automation

Our extension includes a browser automation tool that allows the agent to navigate to URLs, read accessibility trees, and click elements, type text, press keys, scroll, and take screenshots–all controlled programmatically from within a VS Code task via Playwright. We render a live browser preview in a Chromium browser, allowing the developer to watch the agent interact with web applications in real time.

This capability enables use cases that most IDE assistants do not support: verifying a deployed web application after a code change, filling out web forms as part of a testing workflow, scraping documentation to inform a code generation task, or interacting with web-based developer tools (CI dashboards, issue trackers) without leaving the editor.

5.5 Interactive CLI with Slash Commands and `@`-Mentions

The `sorcar` command-line interface runs in two modes. In *interactive* mode (no `-t`/`-f` flag), it presents a Claude-Code-style REPL that connects as a thin terminal client to the same local `sorcar web` daemon used by the VS Code extension. In *non-interactive* mode (`-t` or `-f` supplied), it runs a single task and exits. Either mode honors flags for model selection (`-m`), custom endpoints and headers (`-e`/`-header`), per-task budget caps (`-b`), working-directory pinning (`-w`), worktree isolation (`-worktree`/`-no-worktree`), automatic commit on task finish (`-auto-commit`/`-no-auto-commit`), and toggles for browser tools (`-no-web`) and parallel sub-agents (`-no-parallel`).

The interactive REPL adds two affordances borrowed from modern coding assistants but rare in open-source agents. First, file and folder *`@`-mentions* with ranked project-file completion let the user paste path-aware context into the prompt without leaving the keyboard. Second, a small set of slash commands provides direct control over the agent: `/help` lists every command, `/clear` (alias `/new`) starts a fresh chat, `/resume` reopens a prior chat by id, `/model` (and `/model list`) switches or enumerates models mid-session, `/cost` (aliases `/usage`, `/context`) prints the running token and dollar totals plus context-window utilization, `/skills` and `/mcp` introspect loaded Agent Skills and configured MCP servers, `/autocommit` toggles the worktree's auto-commit policy, `/commands` lists user-defined Markdown slash commands, and `/exit` (alias `/quit`) terminates the session. Custom Markdown slash commands are auto-loaded from `~/.kiss/commands`, `<project>/.kiss/commands`, and the corresponding Claude directories, so users can register reusable prompt templates per-project or globally without modifying the Sorcar source.

5.6 Extensibility: MCP, Skills, and Third-Party Agents

KISS Sorcar exposes three pluggable extension points that let users add capabilities without forking the framework.

Model-Context-Protocol (MCP) servers. The CLI's `sorcar mcp` subcommand registers, lists, inspects, and authenticates MCP servers in either user scope (`~/.kiss/mcp.json`) or project scope (`<project>/.kiss/mcp.json`, with backward compatibility for `<project>/.mcp.json`). Servers may use stdio, HTTP, or SSE transports. An OAuth 2.1 flow (`sorcar mcp auth`) implements dynamic client registration with PKCE and persists tokens under `~/.kiss/mcp_auth/`; a `sorcar mcp debug` command dumps a server's capabilities, tools (with input schemas and granted permissions), resources, and prompts. Discovered tools become first-class `KISSAgent` tools alongside the built-in Bash/Edit/Read/Write tools.

Agent Skills. At startup the framework scans `~/.kiss/skills`, `<project>/.kiss/skills`, Anthropic Claude skill directories, `.agents/skills`, and the bundled Sorcar skills directory. Each skill is exposed to the agent as a callable that the model may invoke when the task description matches the skill's natural-language trigger. This mirrors Anthropic's Claude SKILLS [Anthropic, 2025b] convention so that the same skill files work in both ecosystems.

Third-party messaging and automation agents. KISS Sorcar ships with 23 third-party agents under `src/kiss/agents/third_party_agents` that wrap real-world communication and consumer-product surfaces—BlueBubbles, Discord, Feishu, Gmail, Google Chat, iMessage, IRC, LINE, Matrix, Mattermost, Microsoft Teams, Nextcloud Talk, Nostr, Phone Control, Signal, Slack, SMS, Synology Chat, Telegram, Tlon, Twitch, WhatsApp, and Zalo. It additionally ships a Govee smart-home CLI for controlling IoT lights (on/off, brightness, color, and color temperature) via the Govee Developer API. Each third-party agent declares the credentials or OAuth flow it needs; when missing, the agent attempts to authenticate autonomously and only escalates to the user when a step genuinely requires a human (e.g., entering a two-factor code). This breadth distinguishes KISS Sorcar from Claude Code (whose documented channels include Slack, mobile remote control, and research-preview Telegram/Discord/iMessage but no built-in Gmail, WhatsApp, phone-call, or SMS) and from Cursor (whose Cloud Agent integrations cover Slack and Microsoft Teams).

A fourth extension point—user-curated welcome-screen sample tasks, promptlet injection, and a personal model registry—is described as its own subsection (Section 5.7) because, unlike the three points above, it requires no code change and is exercised by the welcome screen and the input bar of every surface.

5.7 Sample Tasks, Promptlet Injection, and User-Provided Templates, Promptlets, and Models

The welcome screen of the VS Code sidebar, the `sorcar` CLI's empty-prompt view, and the web/mobile surface all greet a new chat with the same list of *sample-task chips*. Each chip is a one-click ready-made prompt; clicking it pastes the prompt into the input bar so the user can edit the angle-bracketed placeholders (`«...»`) before submitting. The bundled sample tasks cover (i) natural-language exploration and revision of an existing workflow (e.g., "Can you show me the detailed step-by-step workflow of `«your algorithm or feature»`"), (ii) authenticating and orchestrating the third-party messaging agents of Section 5.6 (Slack, iMessage, Gmail, SMS, etc.), (iii) scheduling a recurring `kiss-`-prefixed cron job that polls a Slack channel and runs incoming messages as tasks, (iv) adversarial review of an external URL for wrong assumptions, irreproducibility, fraud, evaluation cheating, AI slop, and security vulnerabilities (with a proof-of-concept built and tested by the agent), (v) AI Discovery for the lightest and cheapest model that meets target accuracy/recall on a user dataset using a remote GPU CLI under a fixed dollar budget, (vi) repository optimization through live metric monitoring of a background command, and (vii) the explicit GEPA prompt-optimizer loop with Pareto-frontier bookkeeping. These chips are intentionally written as fully-specified end-to-end prompts—including anti-reward-hacking clauses and report-and-open-in-browser deliverables—so a new user can run a nontrivial Sorcar workflow on the first attempt.

A second customization surface, complementary to sample tasks, is *promptlet injection*. A promptlet (called a "trick" in the implementation) is a short reusable instruction fragment—typically a sentence or short paragraph—that the user wants to splice into the current prompt without retyping. Promptlets are exposed in two ways. The VS Code sidebar renders an "Inject instruction" dropdown next to the input bar populated with every promptlet; selecting one appends it to the current prompt. Across both surfaces, the input bar also offers *ghost-text fast-complete*: as the user types the first few characters of the most recent sentence, a faint suffix proposes the rest of the best-matching promptlet, accepted with Tab. The bundled promptlets capture recurring habits we found useful: "Search internet extensively.",

the test-first habit ("Reproduce the issue by writing integration/end-to-end tests. Then fix the issue."), a pair-with-review habit ("Use claude-opus-4-7 ... ALWAYS use gpt-5.5 (not codex) to carefully and thoroughly review and debug ..."), the anti-reward-hacking reminder, "Build the paper and take screenshots to check and fix formatting.", and a postmortem template ("Why did the last task fail? Thoroughly and precisely analyze the logs and the events of the task. Reproduce the issue ..."). Because promptlets are matched at the start of the current sentence, several promptlets can share a prefix and the dropdown shows all alternatives, so the user picks the right variant from a single keystroke.

The third surface is the *user-provided* side of the previous two—and of the model catalog. KISS Sorcar uses a strict no-clobber policy: bundled defaults are read directly from the package (so an extension upgrade automatically delivers the latest defaults) and user contributions live in three plain-text files under `~/.kiss/` that are auto-seeded once and never overwritten.

- `~/.kiss/MY_TASK_TEMPLATES.md` — the user's welcome-screen chips. Each `## Task` section in the file becomes one chip, and user chips appear *before* the bundled sample tasks so a custom workflow takes precedence on the welcome screen. The file is seeded on first read with a single "Hi!" chip and is never touched again; deleting it triggers a fresh reseed on next launch.
- `~/.kiss/MY_INJECTION.md` — the user's promptlets. Each `## Trick` section in the file is one promptlet, and user promptlets are returned ahead of bundled ones in both the "Inject instruction" dropdown and the ghost-text suggestions, so a user-added promptlet wins on identical prefixes. The file is auto-seeded with a single test-first promptlet on first read.
- `~/.kiss/MY_MODELS.json` — the user's personal model registry, auto-seeded with a documented inert example. Each top-level key is a model id (e.g., `my-org/my-custom-model`) and the value is the same schema used by the bundled `MODEL_INFO.json`: `context_length`, `input_price_per_1M`, `output_price_per_1M`, function-calling/embedding/generation flags, optional cache-pricing overrides, and an optional `thinking` reasoning-effort cap. At import time the loader merges this file on top of the bundled table: matching keys override bundled pricing and context-length entries, and brand-new keys are appended to the catalog. Combined with the `-endpoint`/`-header` controls of Section 5.1, this lets a user front a local vLLM or llama.cpp server (or any OpenAI-compatible HTTP endpoint) as a first-class model in the picker—with correct token accounting, budget enforcement, and tool-calling capability advertised to the agent—without rebuilding or forking KISS Sorcar. Top-level keys beginning with `_` are treated as comments, which is how the seeded example stays inert until the user removes the `_example/` prefix.

The three files are scope-uniform: they live in the same `~/.kiss/` directory used by MCP configuration (`mcp.json`), MCP OAuth tokens (`mcp_auth/`), the persistence DB, and per-user Markdown slash commands (`commands/`, Section 5.5), so a user's entire customization profile is a single self-contained directory that can be version-controlled, shared across machines, or scoped per-project by placing equivalents under `<project>/.kiss/`.

6 The System Prompt

The system prompt is a structured document that governs the agent's behavior across all tasks. It is not a generic instruction to "be helpful" but a specification of engineering practices.

XML-tagged structure. The prompt is organized using XML tags that delimit each concern: `<identity>`, `<visibility_constraint>`, `<tool_rules>`, `<web_research>`, `<code_style>`, `<workflow>`, `<testing>`, `<pre_finish_verification>`, and `<sorcar_specific>`. All three major LLM providers—Anthropic [Anthropic, 2025a], OpenAI [OpenAI, 2025b], and Google [Google, 2025a]—recommend XML tags for structuring complex prompts: they create unambiguous section boundaries that models parse as structural markers rather than content, reducing the chance that the model misinterprets an instruction from one section as applying to another.

Front-loaded engineering rules. We discuss the most opinionated parts of the system prompt first—planning for complex tasks and the testing discipline—and then walk through identity, tool, and workflow rules in roughly the order they appear in `SYSTEM.md`. This ordering reflects the fact that planning and testing are the rules most likely to be ignored when an agent is under pressure to produce a final answer, so we surface them first in the paper even though the file itself opens with the identity block.

We describe the key sections below.

6.1 Planning for Complex Tasks

The planning instructions use a complexity threshold—three or more files, cross-module changes, or architectural work—to decide when formal planning is required:

```
## Complex Task Planning

For work spanning 3+ files, crossing module boundaries,
or changing architecture:

1. List every file to change and why.
2. State the exact intended change per file.
3. Identify dependencies and execution order.
4. State the verification method per change.

Skip this planning step for simple single-file
modifications.
```

Each planning step targets a specific failure mode:

"List files to change and why." This forces the model to enumerate the full blast radius of a change before touching any file. Without this step, the model often discovers mid-task that additional files need changes, leading to incomplete or inconsistent modifications.

"State exact intended change per file." Listing files alone is insufficient; the model must also articulate *what* will change in each file. This converts a vague plan ("update the database module") into a concrete specification ("add a `cache_ttl` parameter to `DatabaseClient.__init__`, modify the query method to check the cache before hitting the database, add a cache invalidation method").

"Identify dependencies and execution order." Some changes must precede others: a new utility function must be written before callers can import it, a migration must run before code that depends on the new schema. Identifying these dependencies prevents the model from applying changes in an order that produces intermediate states where the code does not compile, or tests do not pass.

"State verification method per change." The verification requirement from the Deep Work section is reinforced here at the planning stage, ensuring that verification is planned alongside the changes rather than treated as an afterthought.

The escape clause—"Skip for simple single-file tasks"—avoids the overhead of planning trivial changes. Requiring a formal plan for a one-line typo fix would waste tokens and slow down the agent without any compensating benefit.

6.2 Testing Instructions

The testing section is perhaps the most opinionated:

```
## Testing

- Run lint and typecheckers; fix all errors including
  pre-existing ones.
- Aim for 100% branch coverage on new and modified code.
- Write end-to-end tests only. Do not use mocks,
  patches, fakes, or test doubles. Each test must be
  independent and verify actual behavior.
- **DO NOT** write structural tests which assert on
  the source code.
- After modifications, run only the impacted tests.
- To confirm race conditions: add a random sleep (<0.1s)
  before the suspected racing statements.
- **CRITICAL**: Before running all tests or tests in
  a folder, split the set of tests equally by the
  number of test methods into number of cores - 2 and
  run all splits in parallel using run_parallel tool.
```

Each testing instruction addresses a specific concern:

"Run lint and typecheckers; fix all errors including pre-existing ones." Before committing any change, the agent must ensure it does not introduce lint violations or type errors. This catches a broad class of issues—unused imports, type mismatches, style violations—that would otherwise accumulate across tasks. The clause "including pre-existing ones" prevents the model from rationalizing existing errors as "not my problem" and calling `finish` with a passing result despite a broken build. The instruction makes the agent responsible for the entire codebase health, not just the delta it introduced.

"Aim for 100% branch coverage on new and modified code." LLMs tend to write happy-path tests that cover the main code path but ignore error handling, edge cases, and early-return branches. The 100% target forces the model to write tests for every branch, including error paths and boundary conditions. Moreover, such tests help with regression—developers can use AI coding agents with less risk that changes will break existing program behavior. The wording "aim for" rather than "achieve" acknowledges that perfect coverage is not always feasible, while still setting an ambitious target.

"Write end-to-end tests only. Do not use mocks, patches, fakes, or test doubles." This is the most opinionated rule. Mock tests that verify code calls certain methods in a certain order test the implementation, not the behavior. A test suite built on mocks can pass with flying colors while the system is fundamentally broken, because the mocks hide the real dependencies. End-to-end tests that exercise actual behavior are more expensive to run but provide stronger evidence that the system works. Moreover, writing end-to-end tests forces the model to reason about the system's actual dependencies, often enabling the agent to find additional bugs. The distinction between unit and end-to-end tests matters: a unit test in isolation may verify that a function produces the right output for a given input, but an end-to-end test verifies that the function works correctly within the larger system—with real file I/O, real database connections, and real inter-module interactions.

"Each test independent, verifying actual behavior." Test independence means that running tests in any order produces the same results. Tests that depend on shared state or execution order are brittle and difficult to debug when they fail. "Verifying actual behavior" reiterates that tests should assert on observable outcomes (return values, side effects, system state) rather than implementation details.

"Only run impacted tests after modifications." Running the full test suite after every small change is wasteful when only a few modules are affected. For a large project, a full test run may take minutes, and doing it after every edit adds up to significant wasted time and compute. This instruction directs the model to identify which tests are affected by its changes and run only those, improving iteration speed.

"To confirm races: add random sleep (<0.1s) before racing statements." Race conditions are notoriously difficult to reproduce because they depend on precise timing. By inserting small sleep delays at strategic points, the model can widen the race window and make the bug manifest deterministically during testing. The 0.1-second upper bound keeps the test fast while still being sufficient to expose most races.

"Do not write structural tests which assert on the source code." LLMs frequently fall back on *structural* assertions when they cannot easily exercise a behavior: asserting that a particular function exists, that a class has a certain attribute, that an import appears in a specific order, or that a file contains a specific substring. Such tests do not exercise the system—they merely encode the current shape of the code. They fail spuriously after harmless refactors (renaming a private helper, reordering imports, inlining a function) while providing no evidence that the program actually works. Worse, they create a false sense of coverage: a green suite that consists mostly of structural assertions can coexist with a completely broken runtime. The explicit prohibition steers the agent back to behavioral assertions that survive refactoring and genuinely guard correctness.

Parallel test execution (CRITICAL). Running an entire test suite—or even a whole folder of tests—serially dominates the agent's iteration time and inflates token cost (because the model waits, then re-reads, then re-reasons about a long output). The instruction tells the agent to always partition the test set evenly into *number of cores* $-$ *2* shards and dispatch them concurrently via the `run_parallel` tool whenever it is about to run all tests or all tests in a folder. The "cores $-$ 2" reservation deliberately leaves headroom for the agent's own process and for the IDE so that aggressive parallelism does not starve the foreground experience. We deliberately removed an earlier threshold that gated the rule on having more than 100 tests: in practice the threshold was easy for the agent to under-estimate, the orchestration overhead of `run_parallel` is small enough that parallelization is essentially never harmful for any whole-suite or whole-folder run, and a single unconditional rule is more reliable than

a conditional one. This rule is marked CRITICAL because, like the lint/typecheck obligation, agents otherwise revert to the path of least resistance—a single `pytest` invocation—and pay a large hidden cost on every long-running task.

6.3 Identity and Visibility

The prompt opens with two XML-tagged sections that establish who the agent is and how it communicates with the user:

```
<identity>
You are KISS Sorcar, an AI General Assistant and IDE
developed by Koushik Sen (ksen@berkeley.edu).
Repo: https://github.com/ksenxx/kiss_ai
Version: 2026.6.31

Your sole goal is completing the user's task accurately
and thoroughly. Be rigorous, check facts, and produce
high-quality work.
</identity>

<visibility_constraint>
The user cannot see your thoughts, reasoning, scratchpad,
intermediate tool outputs, or assistant prose. The ONLY
thing the user sees is the string you pass to
finish(summary=...). Compose the full detailed answer
directly inside the summary string of finish(). When
answering informational questions, include the complete
answer in the summary, not a meta-description of what
was done.

**Bad** (meta-description): "Greeted the user and asked
what they'd like to work on. Awaiting a specific task."
**Good** (actual content): "Hi! I'm KISS Sorcar, ready
to help. What would you like to work on?"

The summary must contain the actual content the user
should see, not a third-person narration of what
happened.
</visibility_constraint>
```

Each section addresses a distinct concern:

Identity placement. The `<identity>` block appears first in the prompt, before any behavioral rules. This follows the recommended prompt ordering for frontier models [Anthropic, 2025a, OpenAI, 2025b]: the model should know *what it is* before learning *what to do*. The identity block also consolidates directives that were previously scattered as aggressive imperatives ("BE RELENTLESS," "BE RIGOROUS," "CHECK FACTS," "NO AI SLOP") into a single calm sentence: "Be rigorous, check facts, and produce high-quality work." Research from all three major providers indicates that positive, explanatory framing is more reliable than capitalized commands with frontier models.

Task focus. "Your sole goal is completing the user's task accurately and thoroughly" anchors the model on the task at hand and discourages meta-commentary, tangential exploration, and unsolicited clarification questions that consume tokens without making progress. It also instructs the model to treat errors as obstacles to overcome rather than reasons to stop.

Visibility constraint as a separate section. The `<visibility_constraint>` block is separated from tool rules because it governs a different concern: not *how* to use tools, but *what the user can see*. Without this instruction, the model may "tell" the user something in an intermediate message and then assume the user has seen it, leading to confusion when the user asks for information the model believes it already provided. The clause "not a meta-description of what was done" prevents the model from returning vague summaries like "Fixed the bug in Y" instead of showing the actual fix; it forces the model to include concrete details, results, and outputs in the summary. A second audit found that 3 out of 91 production tasks still produced meta-descriptions (e.g., "Greeted the user and asked what they'd like to work on" instead of the actual greeting). The current version now includes explicit **Bad/Good** examples directly in the prompt—contrasting a third-person meta-description against the actual greeting the user should see—to make the distinction concrete and unambiguous, and closes with the directive that "the summary must contain the actual content the user should see, not a third-person narration of what happened."

6.4 Tool Rules

Tool usage rules are explicit and mechanical:

```
<tool_rules>
## Tool Usage

- Use Write() for new files; Edit() for small changes.
- Use run_parallel() to run parallel tasks and to run
  a sub-task.
- Run Bash synchronously with timeout_seconds (default
  120s). On timeout, retry with a higher value. For
  commands exceeding 10 minutes, run in background,
  redirect output to a file, and poll periodically.
- Use go_to_url() for browser navigation.
- Read large files in chunks.
- **Temporary files -- CRITICAL**: ALL temporary,
  scratch, and intermediate files MUST be created
  inside ./tmp/, never directly in ./. This includes
  research notes, file-information dumps, downloaded
  artifacts, build outputs, and any other transient
  file. Create ./tmp/ if it doesn't exist. Before
  calling finish(), delete every temporary file you
  created in ./tmp/ (but not the directory itself if
  it was pre-existing).
- When multiple independent tool calls are needed, make
  them all in the same turn to maximize parallelism.
  When calls depend on prior results, sequence them
  across turns.

## Context and Continuation

- If running out of context or steps, do not rush. Call
  finish(is_continue=True) to pause and resume the task
  in a new context.
</tool_rules>

- If there is ambiguity or under specification in the
  user task, search the internet to find the most
  reliable and modern solution to resolve the ambiguity.
```

Each tool rule addresses a specific failure mode. An earlier revision of this section opened with an explicit definition—"**PWD denotes current working directory** and does not refer to a directory named PWD"—added after we observed the model creating a literal `PWD/` subdirectory inside the workspace. The current revision drops the disambiguation entirely by replacing every occurrence of `PWD/` in the prompt with the shell-conventional relative form `./` (e.g., `./tmp/`, `./SORCAR.md`), which frontier models interpret unambiguously as the working directory. This is a small example of a recurring simplification pattern: when a prompt rule exists solely to disambiguate a confusing notation, replacing the notation is preferable to explaining it.

"Use Write() for new files; Edit() for small changes." Without this distinction, the model may use `Write()` to overwrite an existing file with a slightly modified version, losing content it forgot to include. By reserving `Write()` for new files and requiring `Edit()` for modifications, the instruction ensures that changes are surgical and that unchanged portions of a file are never at risk.

Bash timeout guidance. LLMs frequently launch shell commands without considering their runtime. A compilation or test suite that takes five minutes will time out at the default 30-second shell timeout in most agent frameworks, causing spurious failures. The instruction to use 120 seconds as the default, retry with higher timeouts on timeouts, and run long-running commands in the background with output redirected to a file provides a mechanical protocol that handles common cases without requiring the model to estimate runtime from first principles.

"Use go_to_url() for browser navigation." The agent has access to multiple tools that could plausibly interact with the web (shell-based `curl`, a Python program, the browser tool, etc.). This clause eliminates ambiguity by specifying which tool to use for browser-based interactions.

"Read large files in chunks." Reading a 10,000-line file in a single tool call consumes a large fraction of the context window. By instructing the model to read files in chunks, the prompt prevents context window exhaustion caused by a single-file read, preserving capacity for the rest of the task.

Temporary files (CRITICAL). A separate, emphatically marked clause requires that *all* temporary, scratch, and intermediate files—research notes, file-information dumps, downloaded artifacts, build outputs—live inside `./tmp/` (the `tmp/` subdirectory of the working directory) and never directly in the working directory. Without this directive, the model creates temporary files in unpredictable locations (the system `/tmp`, the home directory, or scattered throughout the project) or sometimes directly in the project root, polluting the working tree with artifacts that are difficult to distinguish from legitimate project files. Centralizing temporary files in a known directory makes cleanup mechanical and predictable. The companion clause requires the agent to create `./tmp/` if it does not exist and to delete every temporary file it created before calling `finish`, while preserving the `./tmp/` directory itself if it was pre-existing—a deliberately narrow cleanup contract that prevents the agent from accidentally deleting unrelated files that happened to be in the same directory.

Ambiguity protocol. A trailing instruction outside the `<tool_rules>` block—"If there is ambiguity or under specification in the user task, search the internet to find the most reliable and modern solution to resolve the ambiguity"—channels the model's natural tendency to confabulate when underspecified into the structured web-research workflow described below. Rather than letting the model guess (the default behavior) or stop and ask the user (which interrupts the workflow), this directive turns ambiguity into a research task that produces an auditable artifact.

Parallel tool calls. The instruction to "make them all in the same turn" when tool calls are independent reduces latency by allowing the framework to execute multiple tool calls concurrently. Without this instruction, the model tends to issue one tool call per turn even when the calls are independent, wasting round trips.

Context and continuation. When the context window is nearly full, LLMs exhibit a "rush to finish" behavior: they skip verification steps, make hasty edits, and call `finish` with an incomplete result. The continuation instructions redirect that urgency into the continuation protocol (Section 2.2), ensuring that a clean handoff to a new sub-session produces better results than a frantic attempt to squeeze everything into the remaining tokens.

6.5 Pre-flight Checks

We begin every task with mandatory reads, then enforce a read-before-modify discipline:

```
## Mandatory First Actions for project related
tasks -- CRITICAL

**Your VERY FIRST tool call** in every task MUST be
Read("./SORCAR.md") and follow the instructions
in SORCAR.md with highest priority.

## Pre-flight Checks

**Read before modify rule -- NON-NEGOTIABLE**: You MUST
call Read(file_path) on every file BEFORE calling
Edit(file_path) on it. Never Edit a file you have not
Read in the current session.

**Use the file tools, never shell substitutes --
CRITICAL**: To VIEW the contents of any file, you MUST
use the Read() tool. It is FORBIDDEN to inspect or dump
file contents through Bash using cat, sed -n, head,
tail, awk, more, less, nl, grep over a whole file,
echo "$(<file)", or a for-loop of cat. These do NOT
satisfy the read-before-modify rule and waste context
-- always call Read() instead (use max_lines/chunking
for big files). To MODIFY any file, you MUST use the
Edit() or Write() tools. It is FORBIDDEN to edit files
in place through Bash using sed -i, perl -i,
awk ... > file, tee, or output redirection (>, >>) onto
a source/tracked file. Bash may still be used for
non-file-content operations (running tests, ls, grep -l
to find files, git, builds, moving/removing files).
Editing a file you only viewed via a forbidden shell
command is a double violation: you must Read() it first,
then Edit()/Write() it.

Read relevant source files when the task depends on
existing architecture. If referenced files, commands, or
config don't exist, stop and ask the user rather than
guessing.
```

```
**When fixing bugs, issues, or race conditions: write an
end-to-end test that reproduces the problem first, then
fix the code, then verify the test passes.**
```

The "Mandatory First Actions" section ensures that project-specific overrides are loaded before any work begins. This section has undergone several rounds of empirical refinement. In an earlier version that merely mentioned the read inside a separate "Self-Improvement Loop" section, the agent read `SORCAR.md` as its first tool call only 46% of the time. After promoting the read to a dedicated section with imperative language, compliance improved in integration tests but regressed in production: analysis of 91 real tasks showed that 92% never read `SORCAR.md`. Two evasion patterns emerged: (1) the agent used `Bash("cat ...")` instead of the `Read()` tool, and (2) the agent combined the file read with other commands in a single `Bash` call. Explicit prohibitions and ordinal position mandates raised compliance to 100% in integration tests.

The current version collapses the mandatory first action to a single read of `SORCAR.md`, accompanied by the rider "follow the instructions in SORCAR.md with highest priority." This is a deliberate simplification: `SORCAR.md` is the per-repository override file that, when present and non-empty, can extend or override the general system prompt, and loading it first lets the rest of the task be interpreted under the active project's rules. The canonical KISS Sorcar repository ships `SORCAR.md` as an empty placeholder so that no override content is hardcoded inside the framework; each user repository may populate it with whatever project-specific instructions are appropriate. Earlier revisions of the prompt also mandated a second read of a per-project `USER_PREFS.md` preferences file paired with a "Self-Improvement Loop" section that asked the agent to update the file at task end; both have since been removed because the self-learning store accumulated stale project facts (Section 6.8) that drifted out of sync with the evolving code, and the read-only half of the protocol added prompt bulk without measurable benefit. The prohibitions against `Bash`-based reading have since been promoted into an explicit, enumerated rule of their own—"Use the file tools, never shell substitutes"—in the Pre-flight Checks section.

Each pre-flight check targets a specific category of avoidable error:

"Read before modify rule." The most common source of agent-introduced bugs is modifying a file based on an incorrect assumption about its current contents. The model may "remember" an older version of the file from its training data, or it may extrapolate from a partial reading. By requiring a fresh read immediately before any edit, the instruction ensures that the model operates on the file's current state rather than a stale mental model. The companion clause "Read relevant sources if the task depends on existing architecture" extends this rule from individual files to architectural context. A task like "add a caching layer to the database module" requires understanding not just the file to be modified, but also how callers interact with it, what interfaces it provides, and what invariants it assumes. The instruction prevents the model from jumping straight to code generation without understanding the broader context.

"Use the file tools, never shell substitutes." The read-before-modify discipline is only effective if the model actually routes file access through the `Read()`, `Edit()`, and `Write()` tools, where the framework can track, render, and (for edits) verify the operation. In practice, the model repeatedly evaded this by reaching for the shell instead—dumping a file with `cat`, `sed -n`, `head`, `tail`, `awk`, `nl`, or a `grep` over the whole file, and editing in place with `sed -i`, `perl -i`, `tee`, or output redirection (`>`, `>>`). These shell substitutes bypass the read tracking entirely (so a subsequent `Edit()` fires against a file the framework never saw being read) and waste context by spilling raw file contents into the transcript. The current prompt therefore enumerates the forbidden commands explicitly for both viewing and modifying, clarifies that editing a file viewed only through a forbidden shell command is a "double violation" (the file must be `Read()` first, then `Edit()`/`Write()`), and—crucially—carves out the legitimate uses of Bash that must remain available (running tests, `ls`, `grep -l` to locate files, `git`, builds, and moving or removing files). Enumerating both the prohibited and the permitted shell uses prevents the model from over-applying the rule and abandoning Bash for tasks where it is the correct tool.

"If referenced files, commands, or config don't exist, stop and ask the user rather than guessing." LLMs have a strong tendency to confabulate: when asked to modify a file that does not exist, the model will often proceed as if it does, producing edits against phantom content. This instruction

converts a silent failure (incorrect edits applied to a nonexistent file, which silently creates it) into an explicit clarification request.

"Write an end-to-end test that reproduces the problem first, then fix the code, then verify the test passes." This instruction mandates a test-first discipline for bug fixes, specifying the three-step sequence explicitly. The motivation is two-fold: first, a test that reproduces the bug provides concrete verification that the fix is correct (the test should pass after the fix and fail before it). Second, writing the test forces the model to understand the bug precisely before attempting a fix, reducing the risk of an ad hoc patch that addresses a symptom rather than the root cause. Specifying "end-to-end test" rather than just "test" reinforces the no-mocks testing discipline described in Section 6.

6.6 Code Style Guidelines

The prompt encodes a minimalist code philosophy:

```
## Code Style

Write simple, clean, readable code with minimal
indirection. These rules exist because over-abstracted
code is harder to debug and maintain.

- Organize code across multiple files grouped by
  functionality.
- Prefer named functions, classes, and module-level
  helpers over closures and lambdas. Closures obscure
  control flow; use explicit parameter passing instead.
- Eliminate unnecessary attributes, locals, config vars,
  tight coupling, and attribute redirections.
- Eliminate redundant abstractions and duplicate code.
- Public methods must have full docstrings.
- Fix root causes, not symptoms. Before writing code,
  ask: is this simple, elegant, general, and minimal?
- Write documentation only when the task explicitly
  requires it.
```

Each guideline addresses a specific anti-pattern commonly exhibited by LLM-generated code:

"These rules exist because over-abstracted code is harder to debug and maintain." This rationale sentence is deliberate. Anthropic's prompting guide recommends providing motivation behind instructions to help the model generalize correctly [Anthropic, 2025a]. When the model understands *why* a rule exists, it can apply the underlying principle to edge cases that the rule does not explicitly cover.

"Write simple, clean, readable code with minimal indirection. Organize code across multiple files grouped by functionality." LLMs tend to over-engineer solutions, introducing unnecessary abstractions, helper classes, and levels of indirection. Simple code is easier to review, test, and maintain. LLMs also often pile new code onto whichever file is currently being edited, producing 2,000-line modules that conflate unrelated concerns. This directive nudges the model toward a modular layout in which each file has a single, coherent responsibility.

"Prefer named functions, classes, and module-level helpers over closures and lambdas. Closures obscure control flow; use explicit parameter passing instead." LLMs reach for closures whenever a small piece of state needs to be carried alongside a function—producing nested `def`s that capture mutable variables from the enclosing scope. Such closures are difficult to test in isolation, opaque to type checkers, and a frequent source of subtle bugs due to the late binding of captured variables. Rather than a blanket prohibition (as in the earlier version of this prompt), the revised instruction uses positive framing: it tells the model what to *prefer* and explains *why*, which is more effective with frontier models that interpret instructions literally. The instruction steers the model toward explicit data structures (plain functions with arguments, classes with attributes), which are easier to reason about, easier to test, and play well with our no-mocks testing discipline.

"Eliminate unnecessary attributes, locals, config vars, tight coupling, and attribute redirections." LLMs frequently introduce intermediate variables that serve no purpose—for example, assigning a return value to a local variable only to immediately return it on the next line, or storing a constant in a configuration file when it is used in exactly one place. When the model adds a feature that touches multiple files, it may introduce imports, shared global state, or cross-module function calls that create

tight coupling. An attribute redirection occurs when an object stores a reference to another object solely to forward method calls to it—for example, `self.x = other.x` at construction time, creating two paths to the same value. This single consolidated rule addresses all of these anti-patterns.

"Eliminate redundant abstractions and duplicate code." LLMs sometimes create utility functions or classes that duplicate existing functionality; this instruction reminds the model to check for existing implementations before creating new ones.

"Public methods must have full docstrings." While the prompt generally discourages unnecessary documentation (see the last item), public methods are the API surface that other developers and modules depend on. Documentation on public methods is not optional—it specifies the contract.

"Fix root causes, not symptoms. Before writing code, ask: is this simple, elegant, general, and minimal?" LLMs frequently apply symptom-level fixes: adding a null check where the real problem is that a variable should never be null, or catching an exception where the real problem is that the caller passes invalid arguments. This instruction forces the model to trace the causal chain to the root and fix it there. The companion metacognitive instruction asks the model to pause and evaluate its plan before committing to an implementation, spending more inference-time compute on design and reducing the likelihood of producing an unnecessarily complex first draft.

"Write documentation only when the task explicitly requires it." Claude Opus 4.6 tends to generate many documentation files. This instruction prevents the behavior.

6.7 Deep Work Rules

```
## Deep Work

- For tasks involving "align", "match", or "make
  consistent": read the target state fully before
  editing. Never edit based on vague recollection.
- Use concrete values, not indirections. Read file Y
  first, then write the specific values into file X.
- List concrete planned changes before executing
  multi-part work.
- Every meaningful change needs a concrete verification
  method (test, grep, CLI check).
```

The deep work rules address a failure mode where the model interprets an instruction loosely and makes changes that are directionally correct but concretely wrong:

"For 'align'/'match'/'make consistent': read the target state before editing." When a user says "make file A consistent with file B," the model often reads file A, infers what file B probably contains, and edits A based on that inference—without ever reading B. This instruction mandates reading the target first, ensuring that the alignment is based on concrete facts rather than assumptions.

"Use concrete values, not indirections (read Y first, then write specific values into X)." A related failure mode occurs when the model's plan says "update X to match Y" but the model never resolves what Y actually is. The instruction requires the model to first read Y, extract the specific values, and then write those values into X. This eliminates a class of errors where the model's mental model of Y differs from reality.

"List concrete planned changes before executing multi-part work." When a task requires changes to multiple files, executing them one at a time without a plan leads to inconsistencies: the model may change a function signature in one file but forget to update a caller in another. Listing all planned changes before executing any of them forces the model to consider the full scope of the change and identify dependencies.

"Every meaningful change needs a concrete verification method." A change without a verification method is a change that cannot be confirmed to work. This instruction requires the model to pair each change with a specific check—a test, a grep for the expected pattern, a CLI command that exercises the changed behavior—ensuring that the change can be validated programmatically rather than by visual inspection of a diff.

6.8 Self-Improvement via the Framework

Earlier revisions of `SYSTEM.md` contained an explicit *Self-Improvement Loop* section that instructed the agent, before calling `finish`, to update a `USER_PREFS.md` file with any reusable user preference, project convention, file location, or other durable fact discovered during the task. The section also encoded the curation rules: no code snippets or symbol names in entries, skip one-off task details, remove conflicting older entries, and keep the file small because its content was loaded into every subsequent task's context. The motivation was that the preferences file would let the agent accumulate project knowledge across sessions even though each session starts with a fresh context window.

The current `SYSTEM.md` no longer contains a `## Self-Improvement Loop` section and the `USER_PREFS.md` file has been removed from the project entirely. The mechanism was removed because the self-learning store accumulated *stale* information about the project. Facts that were true when an entry was written—file locations, API endpoints, project conventions, and design decisions—silently went out of date as the codebase evolved, and because the stored entries were injected into every subsequent task's context, later tasks acted on outdated assumptions. In several cases this caused the agent to reintroduce bugs that had already been fixed: it "learned" a workaround or invariant, that fact later became false, and the agent kept applying the obsolete knowledge. The preferences file thus accumulated noise faster than useful invariants and actively regressed the code rather than improving it. Project-specific instructions turned out to be better captured in the per-repository `SORCAR.md` override file, which the user controls directly and keeps current. Removing the self-learning mechanism entirely shrinks the prompt, eliminates a brittle cross-session state file that drifted out of sync with the code, and concentrates project knowledge in a single human-curated place that cannot silently go stale.

6.9 Pre-Finish Verification

Before declaring a task complete, the agent must pass a structured verification checklist:

```
## Pre-Finish Verification -- CRITICAL

Before calling finish(success=True):

1. Re-read and verify every modified file.
2. If you created or modified ANY .py, .ts, .js, .css,
   .tsx, or .jsx file in this session: you MUST run
   uv run check --full and fix all errors. This is not
   optional. Do NOT call finish without running this
   command first. If the project doesn't use uv, run
   the equivalent lint/typecheck command.
3. Check each user requirement against what was
   delivered.
4. **Clean up temporary files -- MANDATORY**: You MUST
   delete every temporary file you created in ./tmp/
   during this session (research notes, information-*.md,
   file-information-*.md, scratch scripts, downloaded
   artifacts, etc.). Explicitly run Bash("rm -f
   ./tmp/<each-file-you-created>") and then
   Bash("ls ./tmp") to confirm they are gone. Do NOT
   call finish(success=True) while any temp file you
   created still remains. Do NOT delete files you did
   not create.
5. If any check fails, keep working.
6. After 3 failed retries of the same fix approach,
   step back and rethink from scratch.
```

Each step in this checklist addresses a specific way agents declare premature success:

"Re-read and verify every modified file." This is the analog of a code review performed by the agent on its own work. The model may have introduced a typo, forgotten to close a bracket, or made an edit that looked correct in the diff but was wrong in the full-file context. Re-reading the file after all edits are complete catches these errors.

"If you created or modified ANY code file: you MUST run uv run check." The original instruction ("Run required checks; fix failures") was too vague: analysis of 91 real tasks showed that 70% of code-modification tasks (7/10) skipped lint/typecheck entirely. The revised version enumerates the exact file extensions that trigger the obligation and names the exact command to run, converting a

soft guideline into an unambiguous, verifiable mandate. A trailing fallback clause—"If the project doesn't use uv, run the equivalent lint/typecheck command"—generalizes the rule to projects with different toolchains while keeping `uv run check --full` as the canonical instruction.

"Check each user requirement against delivery." The model may have completed a task that it *thinks* satisfies the user's request, but actually misses a requirement. This instruction forces a systematic comparison between the original task description and the delivered result, catching gaps and misinterpretations.

"Clean up temporary files." The temporary-files directive in the Tool Usage section requires the agent to write scratch files into `./tmp/`, but without an explicit cleanup step at the end of the task, those artifacts persist and pollute the project's working tree over time. This step mandates deletion of every temporary file the agent created during the session. The companion clause "Do NOT delete files you did not create" is essential: the `tmp/` directory may contain artifacts from other sessions or from the developer's own scratch work, and an indiscriminate `rm -rf tmp/*` would destroy unrelated content. Together, the two clauses form a narrow, audit-friendly cleanup contract. The current revision additionally embeds an explicit command template—`Bash("rm -f ./tmp/<files-you-created>")`—directly inside the bullet, so the agent has a concrete, copy-pasteable cleanup command and does not have to invent its own (occasionally over-broad) deletion strategy. It further requires the agent to run `Bash("ls ./tmp")` afterward to confirm the files are gone, and forbids calling `finish(success=True)` while any agent-created temporary file remains—turning the cleanup from an easily skipped suggestion into a verified post-condition.

"If any check fails, keep working." Without this instruction, the model may call `finish(success=True)` even when it knows a check has failed, rationalizing that the failure is "minor" or "unrelated." The instruction makes the rule absolute: no finishing until all checks pass.

"After 3 failed retries of same fix, rethink from scratch." LLMs can enter repetitive loops where they apply the same incorrect fix repeatedly, each time hoping for a different result. The three-retry threshold forces the model to break out of such loops by abandoning the current approach and reconsidering the problem from first principles. This is analogous to the debugging heuristic "if you've been staring at the same code for twenty minutes, you're looking in the wrong place."

6.10 Web Research Protocol

When the agent needs external knowledge, the prompt prescribes a structured research workflow rather than allowing ad-hoc browsing:

```
## Web Research

When a task requires searching the internet, researching
a topic, or answering questions that benefit from current
information:

- Visit at least 10 distinct websites per research
  session. Do not stop early or rationalize visiting
  fewer. **This is a hard requirement -- you MUST
  visit 10 sites, not 5 or 8.**
- You MUST use go_to_url() to visit each site. Do NOT
  use Bash("curl ...") or Bash("wget ...") as a
  substitute for visiting websites. Using curl/wget to
  fetch pages does not count toward the 10-site
  requirement.
- Procedure:
  1. Create ./tmp/information-{unique_id}.md with
     header: # Web Research -- Websites visited: 0/10
  2. Per site visited: (a) use go_to_url() to visit the
     site, (b) extract information needed for the task
     without deep thinking, (c) use Edit() to append
     ## [N/10] URL + extracted information to the file,
     (d) use Edit() to update the header counter from
     N-1 to N. **You must update the counter after each
     site.**
  3. Do not proceed to synthesis until the counter
     reaches 10. **Check the counter -- if it says less
     than 10, keep visiting more sites.**
  4. If results dry up, try different queries, synonyms,
     official docs, GitHub repos/issues, Stack Overflow,
     blogs, Reddit, papers, and API references.
  5. After reaching 10, review all findings and
```

```
    synthesize.
- Ask the user for login help when a page requires
  authentication.

This requirement applies to research and
information-gathering tasks. For pure code edits, bug
fixes, or file modifications where you already have
sufficient context, proceed directly.

**The information file is mandatory.** You MUST create
the ./tmp/information-{unique_id}.md file and track the
counter. Do NOT skip the file and answer from memory.
Do NOT synthesize your answer without first reaching 10
in the counter. The file is your proof of work -- if it
doesn't exist when you call finish, you violated this
rule.

## Real-Time Data -- CRITICAL

For questions about **current events, weather, stock
prices, sports scores, or any time-sensitive
information**: you MUST use tools (go_to_url, Bash) to
look up the data. Do NOT answer from your training data
-- it is outdated and will produce wrong dates, wrong
numbers, and wrong facts.

**Do NOT fabricate or exaggerate source counts.** If
you visited 4 websites, do not claim "10+ sources" or
"extensive research." State the actual number of
sources you consulted.
```

The rationale is a two-phase separation between *collection* and *synthesis*. LLMs tend to anchor on the first few results they encounter, which biases their solutions toward a narrow slice of the design space. By forcing the agent to accumulate a broad set of information into a file *before* reasoning about it, the protocol counteracts anchoring bias and encourages the model to consider diverse approaches. An earlier version of this prompt required visiting at least 30 websites, but the threshold was lowered to 10 after we observed that the 30-site requirement frequently inflated token cost and wall-clock time without proportionally improving answer quality on routine research tasks; 10 sites remain enough to span official documentation, primary sources, secondary commentary, and a few divergent perspectives. The resulting information file serves as an auditable artifact of what the agent considered. The structured procedure with a counter header (`# Web Research -- Websites visited: 0/10`) and per-site entries (`## [N/10] URL`) addresses an empirically observed failure mode in which the model claims to have "visited many sites" after only a handful of fetches; a concrete counter forces the model to verify the actual number visited before declaring the collection phase complete. An additional emphatic clause—"This is a hard requirement – you MUST visit 10 sites, not 5 or 8"—was added after we observed the model rationalizing partial compliance ("I visited 7 sites, which is close enough"). The instruction to try "different queries, synonyms, official docs" when results dry up prevents the agent from giving up prematurely on a narrow set of search terms.

An additional evasion pattern observed in production was the agent using `Bash("curl ...")` to fetch web pages instead of `go_to_url()`, thereby bypassing the browser-based visit counter while still gathering information. Analysis of 91 real tasks found 4 research tasks that visited fewer than the required minimum number of sites (ranging from 1–5 URLs), with one task (1179) using `curl` to fetch content from 15+ sites without triggering `go_to_url`. The explicit prohibition of `curl`/`wget` for research closes this loophole.

A second audit uncovered two additional failure modes. First, the mandatory information file (`tmp/information-{id}.md`) was created in 0 out of 4 research tasks—the agent skipped file creation entirely and answered from memory or from a handful of sites. The current version adds explicit mandatory language: "The file is your proof of work—if it doesn't exist when you call finish, you violated this rule." Second, one task claimed "a synthesis of 30+ sources" in its result summary while having visited only 4 URLs—a fabricated source count. A new "Real-Time Data" section addresses both hallucination (answering current weather, news, or stock questions from stale training data—one task reported news from the wrong year) and source fabrication ("Do NOT fabricate or exaggerate source counts").

The login instruction addresses a practical obstacle in web research: many websites require authentication before revealing their content. Rather than silently skipping gated pages or hallucinating their contents, we instruct the agent to ask the user for help with login.

Scoped applicability. The final paragraph—"This requirement applies to research and information-gathering tasks. For pure code edits, bug fixes, or file modifications where you already have sufficient context, proceed directly"—is an important addition. Without this exemption, the agent would perform 10 website visits even for simple one-line code fixes where the necessary context is already in the file being edited, wasting tokens and tool-call budget. The exemption allows the agent to skip research when the task is purely mechanical, while still enforcing thorough research for tasks that benefit from external information.

6.11 File Browsing Protocol

When a task requires understanding multiple source files before making changes, the prompt prescribes the same two-phase collect-then-synthesize discipline used for web research, but applied to the local file system:

```
## File Browsing

When exploring unfamiliar code, collect information and
code snippets in ./tmp/file-information-{unique_id}.md
as you go relevant for the task, then review the
collected material and think deeply before acting.
```

This instruction addresses a failure mode distinct from the web research case. When an agent must read many project files to understand a codebase before making changes, it tends to read a file, form a hypothesis, and immediately begin editing—anchoring on the first few files it encounters and missing relevant context in files it never opens. Worse, each file read consumes context window tokens; by the time the agent has read enough files to understand the full picture, it may have already spent most of its context window on the raw file contents, leaving little room for reasoning and code generation.

The file browsing protocol counteracts both problems. By writing a structured summary of each file's relevant information into a temporary markdown file, the agent externalizes its understanding into a compact artifact that persists across context boundaries. The instruction to collect "without overthinking" is deliberate: during the collection phase, the agent should extract and record facts (function signatures, class hierarchies, call sites, invariants) rather than analyze or plan. Analysis happens in the second phase, when the agent reads its own summary file and reasons about the collected information as a whole.

This two-phase separation provides three benefits. First, it prevents premature commitment: the agent cannot start editing until it has surveyed the relevant files, reducing the risk of changes that are locally correct but globally inconsistent. Second, the summary file is typically much smaller than the raw source files, freeing up context window capacity for subsequent reasoning and editing phases. Third, the summary file serves as an auditable artifact—the developer can inspect it to verify that the agent considered the right files and extracted the right information before making changes.

6.12 Desktop Application Control

The agent can interact with graphical desktop applications using screenshots, keyboard, and mouse:

```
## Desktop Apps

Interact with desktop applications using screenshots, keyboard, and mouse. Do not launch VS Code or its
     extensions.
```

This instruction enables the agent to operate GUI applications (Preview, browsers, graphical diff tools) when command-line alternatives are insufficient. The explicit prohibition on launching VS Code prevents a recursive loop: since the agent runs *inside* a VS Code extension, launching another VS Code instance or modifying extension state from within the agent could corrupt the host session or create deadlocks. Note that modern LLMs support desktop control abilities, and we are merely exploiting them.

6.13 Sorcar-Specific Overrides

A final section provides project-specific instructions that are injected when the agent operates on its own codebase:

```
## Sorcar-specific

- Lint/typecheck/format: ‘uv run check --full‘. Tests:
  ‘uv run pytest -v‘ (timeout 900s).
- Your SYSTEM.md (the system prompt) is located at
  ~/.vscode/extensions/ksenxx.kiss-sorcar-2026.6.31/
    kiss_project/src/kiss/SYSTEM.md
- KISS Sorcar paper:
  https://github.com/ksenxx/kiss_ai/blob/main/
    papers/kisssorcar/kiss_sorcar.tex
- Third-party agents: kiss/agents/third_party_agents
- Claude SKILLS: kiss/agents/claude_skills. You can
  use them as necessary.
- **If you create any artifact that the user can use
  after the task is over, you MUST create them in a
  directory and add the directory contents to git.**
- MAINTAIN a ./tmp/PROGRESS.md across agent sessions
  logging details of all the steps you have done so far
  from the start with explanation and relevant code
  snippets.
- **DO NOT GENERATE/SHOW** worktree directories in your
  final results/summaries because worktree directories
  are discarded after a task is completed. Rather show
  the directories relative to the main repo.
- Authenticate unauthenticated third-party agents; ask
  the user only when a page requires human
  authentication. You MUST collect any security or
  authentication code or token.
```

These overrides serve eight purposes. First, they specify the exact toolchain commands for the KISS project itself (`uv run check --full`, `uv run pytest -v` with a 900-second timeout), eliminating guesswork about which linter, formatter, or test runner to use. Second, the agent is told the on-disk location of its own `SYSTEM.md` (under the bundled VS Code extension directory, ending in `/SYSTEM.md`) so that questions about its own prompt or behavior can be answered by reading the canonical file rather than from memory. This pointer is paired with a URL to the paper's LaTeX source, giving the agent two complementary self-references: the system prompt for operational rules and the paper for design rationale. Third, the instructions expose a third-party agent integration layer: the agent is told where third-party agents live (`kiss/agents/third_party_agents`). When a third-party agent requires authentication, the agent handles it autonomously and only prompts the user when a page genuinely requires human credentials; the explicit rider "You MUST collect any security or authentication code or token" instructs the agent to elicit and persist any code/token returned during such an interactive auth step so that subsequent calls do not have to re-prompt the user. Fourth, Claude SKILLS [Anthropic, 2025b] are made discoverable at `kiss/agents/claude_skills` (populated at install time from the bundled VS Code extension) and at the standard Anthropic locations `~/.claude/skills` and `<project>/.claude/skills`, with the note "You can use them as necessary," giving the agent access to Anthropic's skill library for common software engineering patterns; the permissive framing (rather than a mandate) lets the agent select skills opportunistically when a task matches one. Fifth, the `SORCAR.md` override mechanism—enforced by the Mandatory First Actions section—is the per-repository override file that, when present and non-empty, can extend or override the general system prompt, forming a hierarchy: general system prompt $\rightarrow$ Sorcar-specific instructions $\rightarrow$ repository-specific `SORCAR.md`. We deliberately ship the canonical repository's `SORCAR.md` as an empty placeholder so that no override content is hardcoded inside the framework; each user repository is free to populate it with whatever project-specific instructions are appropriate, and may further `@include` additional markdown files from `SORCAR.md` for richer per-repository documentation. Sixth, a directory-with-git-commit rule requires that any artifact the user can use after the task ends be created inside a directory and added to git, ensuring that durable outputs are version-controlled and easy to locate rather than scattered as loose files. Seventh, the agent is required to maintain a `./tmp/PROGRESS.md` across sub-sessions logging every step taken so far with explanations and relevant code snippets; this acts as a persistent scratchpad that bridges Relentless Agent continuations and lets a fresh sub-session pick up exactly where the previous one left off without re-deriving prior decisions. Eighth, the worktree-presentation rule ("DO NOT GENERATE/SHOW worktree directories in your final results/summaries... rather show the directories relative to the

main repo") prevents the agent from quoting transient worktree paths that are discarded after merge; user-visible references must point at paths in the main repository so that copy-pasted file names and shell commands remain valid after the task completes.

7 Painless Software Engineering with KISS Sorcar

A central claim of our system is that natural-language interaction can replace manual code inspection and ad hoc scripting for understanding and evolving nontrivial subsystems. While developing KISS Sorcar, we found two recurring workflows particularly useful: *conversational redesign*, in which the developer asks the agent to explain a subsystem and then directs incremental changes in plain English; and *invariant-driven bug discovery and fixing*, in which the developer states a user-visible invariant the system MUST satisfy and asks the agent to reproduce any violation with an end-to-end test before fixing it. We illustrate both with real sessions drawn from the project's own `sorcar.db` task history, reproducing the prompts verbatim and summarizing the agent's responses.

7.1 Conversational Redesign via Natural Language

In this workflow we first ask KISS Sorcar to generate a detailed, step-by-step description of a workflow or algorithm we found buggy, and then we ask KISS Sorcar to revise some of the buggy steps in natural language. We illustrate this with a real development session in which the worktree merge workflow (Section 2.5) was first understood and then redesigned entirely through conversational prompts. The session comprises four consecutive tasks; we reproduce the prompts verbatim and summarize the agent's responses.

Step 1: Understanding the existing workflow. The developer begins by asking the agent to explain the current post-task git lifecycle:

```
Can you tell me what happens, step by step, with git in worktree_sorcar_agent.py when a task finishes?
```

The agent reads the source code and returns a structured summary of the four-phase lifecycle: (1) during `run()`, a new branch and worktree are created and the task executes inside the worktree; when the task completes, *nothing is committed or merged*—the result is returned with merge instructions appended and the worktree stays pending; (2) `merge()` calls `_finalize_worktree()`, which stages all changes, generates a commit message via the LLM, commits, removes the worktree, checks out the original branch, and runs `git merge`; (3) `discard()` removes the worktree, prunes, checks out the original branch, and deletes the task branch; (4) in CLI mode an interactive prompt forces the user to choose `[c]ommit and merge` or `[d]iscard` before exiting. The agent also notes a key design invariant: nothing auto-merges; auto-commit occurs only at merge/finalize time; all steps are idempotent; and state can be recovered from git on restart.

Step 2: Simplifying the workflow via natural language. Armed with the workflow description, the developer decides the three-way choice (auto-merge, manual merge, discard) is unnecessarily complex and issues a redesign request:

```
Can you change worktree_sorcar_agent.py and the extension so that after the agent
finishes its task, it simply asks "Commit and Merge" or "Discard". When "Commit
and Merge" is clicked by the user, the agent must commit the changes with a
generated commit message, merge the branch with the original branch, and delete
the worktree and the branch associated with the worktree. If the user clicks
"discard", it must delete the worktree and the branch, and checkout the
original branch.
```

The agent modifies six files across Python and TypeScript: it updates `discard()` to check out the original branch before deleting the task branch, removes the `manual_merge()` method entirely, simplifies `merge_instructions()` to show only two options, updates the webview UI to replace the three-button toolbar with a two-button "Commit and Merge or Discard?" bar, removes the `manual` action type from the TypeScript type definitions, and removes the corresponding handler from the Python backend. Three tests for the deleted manual-merge path have been removed, and one routing test has been updated. All 28 worktree tests pass after the change.

Step 3: Investigating unexpected post-merge state. After testing the simplified flow, the developer notices that files appear in the Source Control panel of VS Code on the original branch after pressing "Commit and Merge" and asks:

```
After the user presses "Commit and Merge", why do the modified files show up as
committed in the original branch?
```

The agent traces the exact execution path through `squash_merge_branch()` and discovers that the implementation deliberately *unstages* the squash-merged changes via `git reset HEAD` so they appear as uncommitted working-tree modifications in VS Code's Source Control panel. The design intention was to let the user review the diff before committing manually. The agent reports this finding along with the relevant code snippet and a verified end state showing no new commits on the branch.

Step 4: Directing a design change in one sentence. The developer realizes that code review already happened while the worktree was active and decides the unstage step is unnecessary:

```
The review is already happening in the worktree branch. You don't need the user
to review the modified and new files in the original branch. Fix it.
```

The agent replaces the `git reset HEAD` call in `squash_merge_branch()` with a conditional `git commit -no-edit` that uses the auto-generated squash message. It adds a `git diff -cached -quiet` guard for the edge case where the merge produces no changes. One existing test (`test_merge_leaves_changes_uncommitted`) is renamed to `test_merge_commits_changes` and its assertions are updated to verify a clean working tree. All 104 worktree tests pass.

Discussion. The four-step session illustrates the development loop that our system supports. The developer never opens a source file, never writes a line of code, and never runs a test manually. Instead, the entire cycle—understand the workflow, redesign it, investigate an anomaly, direct a fix—happens through natural-language prompts, with the agent handling code reading, multi-file editing, test updates, and verification. This style of development becomes possible because of the agent hierarchy described in Section 2: the Worktree Sorcar Agent isolates changes on a branch, the Chat Sorcar Agent preserves conversational context across tasks, and the Relentless Agent automatically continues when the context window is exhausted.

7.2 Invariant-Driven Bug Discovery and Fixing

A second workflow we use frequently is *invariant-driven bug discovery and fixing*. The developer never localizes the defect or even names the suspect file. Instead, the prompt (i) states a user-visible invariant that the system MUST satisfy, (ii) asks the agent to reproduce any violation by writing an end-to-end test before touching the source, (iii) asks for the fix, and (iv) assigns one model to write the test and the fix and a second model from a different vendor to review and debug that work—explicitly asking the reviewer to look for missed call sites and newly introduced bugs. Phrasing the bug as an invariant transfers diagnostic effort to the agent; requiring an executable reproduction before the fix produces an artifact the reviewer model can refute or confirm independently of the test author's narrative; and the cross-vendor split catches mistakes that a single model would systematically miss.

We illustrate the pattern with a recent task drawn verbatim from `sorcar.db`, in which two invariants of the interactive `sorcar` CLI's input box were stated together:

```
in sorcar cli interactive, the user MUST be able to enter multi-line inputs in
the input box. Moreover, all text shown must be word wrapped. Reproduce the
issue by writing an integration test. Then fix the issue. Use claude-opus-4-7
model for all tasks including coding, bug fixing, and test creation. Use gpt-5.5
model (not codex) for thorough review and debugging of the work done by the
other model. Check if the other model has missed some code or has introduced
bugs.
```

The agent executes the workflow in three phases.

Phase 1: Reproduce. Under `claude-opus-4-7` the agent locates the prompt loop in `src/kiss/agents/sorcar/cli_prompt.py`, writes a new end-to-end test file `test_cli_multiline_input.py` containing nine integration tests that drive a real `prompt_toolkit PromptSession` through `create_pipe_input` (the same pattern as the existing `test_at_mention_picker.py`), and confirms the first test fails: typing `"hello"`, then Esc+Enter, then `"world"`, then Enter returns just `"hello"` against the unmodified code, witnessing the multi-line invariant violation. The diagnosis is that `PromptSession` was instantiated without `multiline=True`, so Enter always submitted and the buffer never wrapped.

Phase 2: Fix. Still under `claude-opus-4-7`, the agent passes `multiline=True`, `wrap_lines=True`, and a `prompt_continuation` callable to the session; the continuation callable returns an ANSI string that repaints the cyan `|` left border on every wrapped or continuation row so the framed panel stays visually consistent. An Enter binding filtered by `~completion_is_selected` calls `event.current_buffer.validate_and_handle()` so plain Enter still submits, while `escape,enter` (Alt+Enter), `c-j` (Ctrl+J), and the kitty/foot/WezTerm CSI-u sequence `ESC[13;2u` (matched as a raw-char tuple because `prompt_toolkit` does not pre-map it) insert real newlines. The welcome banner in `cli_repl.py` is updated to document the new keys.

Phase 3: Cross-model review. The agent switches to `gpt-5.5` (not Codex) and re-reads the diff, the surrounding modules, and the new tests, looking specifically for missed call sites and broken edge cases. The review surfaces a real `prompt_toolkit` library limitation that `claude-opus-4-7` had failed to notice: the xterm `modifyOtherKeys` Shift+Enter escape `\x1b[27;2;13~` is pre-mapped to `Keys.ControlM` inside `prompt_toolkit.input.ansi_escape_sequences.ANSI_SEQUENCES`, which means any user binding for it can never fire. A previously-added (always-failing) test asserting that this sequence inserts a newline is replaced with one asserting the documented fall-through behaviour (it submits like plain Enter), and the welcome banner is amended to direct users on those terminals to Alt+Enter or Ctrl+J. The review also corrects ruff E402/I001 import-ordering violations introduced during the fix. Verification runs `uv run pytest -v` on the new file (9/9 pass) and on the neighbouring CLI suites `test_at_mention_picker.py`, `test_cli_repl.py`, and `test_cli_panel.py` (49/49 pass), followed by `uv run check -full` (ruff, mypy, mdformat, syntax, api-docs) before the agent reports completion.

Discussion. The pattern generalizes. Recent entries in `sorcar.db` apply the same five-line template—"*X MUST hold. Reproduce by an end-to-end test. Then fix. Use model A for code, model B for review.*"—to fix invariant violations in the Sorcar CLI's notification stream, in the routing of bash tool outputs to the Result panel, and in syntax-highlighting the output of the `Read` tool. Three properties make the loop effective. First, by phrasing the bug as an invariant rather than as a stack trace, the developer transfers diagnostic effort to the agent without having to localize the defect in advance. Second, requiring an executable end-to-end test *before* the fix produces a reproducible artifact that the reviewer model can refute or confirm independently of the implementer's narrative, and that survives in the test suite as a regression guard. Third, splitting code production and code review across two vendors with different training data catches mistakes that a single model would systematically miss—including library-internal assumptions, such as the `prompt_toolkit` pre-mapping above, that no amount of self-reflection by one model is likely to surface. Dynamic model switching (Section 5) makes both halves of this split a single `set_model()` call inside one continuous task.

8 Related Work

Code-specialized language models. Beyond the general-purpose LLMs that our system can use as backends, a rich line of work has produced models specifically trained for code. Code Llama [Rozière et al., 2023] fine-tunes Llama 2 for code generation and infilling. StarCoder [Li et al., 2023] trains on permissively licensed code from GitHub with a fill-in-the-middle objective. DeepSeek-Coder [Guo et al., 2024] trains on a 2-trillion-token corpus of code and natural language with a repository-level context window. More recently, frontier models have been optimized specifically for agentic software engineering. Claude Opus 4.6 [Anthropic, 2026a] and its successor Opus 4.7 [Anthropic, 2026b] advance long-horizon coding and agentic task execution; OpenAI's GPT 5.5 [OpenAI, 2026b] similarly targets agentic software engineering with strong code generation and tool-use capabilities. Cursor released Composer 2 [Cursor Research, 2026], a custom fine-tuned coding model trained with

large-scale reinforcement learning. Kimi K2.5 [Kimi Team, 2026] is an open-source multimodal agentic model that jointly optimizes text and vision and introduces Agent Swarm for parallel task decomposition. GLM-5.1 [Z.ai, 2026] is a 754-billion-parameter mixture-of-experts model from Z.ai that sustains autonomous coding over multi-hour sessions, achieving state-of-the-art on SWE-Bench Pro. Our system is model-agnostic and can leverage any of these models through its pluggable LLM backend, benefiting from advances in code-specialized pre-training without architectural changes.

Code generation agents. SWE-Agent [Yang et al., 2024b] and OpenHands [Wang et al., 2024b] provide LLM-based agents for resolving software engineering tasks such as GitHub issues. Both use a single-session execution model without automatic continuation. Agentless [Xia et al., 2024] takes the opposite approach, showing that a simple two-phase localize-then-repair pipeline without autonomous agent loops can achieve competitive results on SWE-bench [Jimenez et al., 2024]. Devin [Cognition Labs, 2024], an industrial product marketed as "the first AI software engineer," operates in a sandboxed environment with shell, browser, and editor access. Claude Code [Anthropic, 2025b] is Anthropic's terminal-based agentic coding tool that gives Claude direct access to a shell, file system, and development tools for multi-step engineering tasks. OpenAI's Codex [OpenAI, 2025a] is a cloud-based software engineering agent that executes coding tasks in sandboxed environments with full repository context. Aider [Gauthier, 2023] provides a terminal-based pair programming interface with tight git integration, automatically committing each change. Our Relentless Agent layer addresses the single-session limitation common to most of these systems, while our worktree isolation provides stronger safety guarantees than per-change commits do.

ReAct and tool use. The ReAct framework [Yao et al., 2023b] interleaves reasoning and action. Toolformer [Schick et al., 2023] teaches models to use tools via self-supervised learning. We use native function calling provided by modern LLM APIs rather than in-context tool descriptions, thereby reducing prompt overhead and improving reliability.

Reasoning and planning. Chain-of-thought prompting [Wei et al., 2022] demonstrated that eliciting step-by-step reasoning dramatically improves LLM performance on complex tasks. Tree of Thoughts [Yao et al., 2023a] generalizes this to deliberate search over multiple reasoning paths. Reflexion [Shinn et al., 2023] introduces verbal reinforcement learning, where an agent reflects on failed attempts and produces self-critiques that improve subsequent trials. Our continuation protocol is conceptually related to Reflexion: failed sub-sessions produce summaries that inform subsequent attempts. However, in our case, we use the summaries to continue the task.

Multi-agent software development. ChatDev [Qian et al., 2024] models the software development process as a conversation between role-playing agents (CEO, CTO, programmer, tester) organized in a waterfall-like pipeline. MetaGPT [Hong et al., 2024] takes a meta-programming approach, encoding standard operating procedures as structured outputs that coordinate specialized agents. AutoGen [Wu et al., 2023] provides a general-purpose framework for multi-agent conversation, enabling flexible topologies beyond fixed pipelines. These systems focus on generating entire applications from scratch. We take a different approach: rather than simulating an organization of specialists, we use a single agent with broad access to tools and optional parallel sub-agents for embarrassingly parallel sub-tasks, prioritizing practical utility on real-world codebases over role-playing fidelity.

Multi-turn autonomous agents. AutoGPT [Significant Gravitas, 2023] and BabyAGI [Nakajima, 2023] implement multi-step autonomous agents. These systems typically lack budget controls and safe code isolation. Our layered architecture addresses each of these concerns in a dedicated layer.

Software engineering benchmarks. SWE-bench [Jimenez et al., 2024] evaluates agents on real-world GitHub issues drawn from popular Python repositories, requiring the agent to localize and fix bugs given only the issue description. It has become the de facto standard for measuring agent capabilities in software engineering. HumanEval [Chen et al., 2021] and MBPP [Austin et al., 2021] evaluate function-level code generation from docstrings. LiveCodeBench [Jain et al., 2024] addresses benchmark contamination by continuously collecting fresh competition problems from LeetCode, AtCoder, and CodeForces, and extends evaluation to self-repair, code execution, and test output prediction. These benchmarks focus on isolated coding problems; We target the broader workflow of multi-file, multi-step software engineering tasks that require tool use, testing, and version control.

Agent frameworks and orchestration. LangChain [LangChain, 2022] provides modular abstractions for building LLM applications, including agent loops, tool integration, and memory. It focuses on

composability and breadth of integrations rather than on the specific concerns of software development. DSPy [Khattab et al., 2024] treats LLM calls as declarative modules whose prompts and few-shot examples are compiled and optimized automatically, enabling systematic improvement of multi-stage pipelines; it targets prompt optimization rather than end-to-end software engineering workflows. CrewAI [CrewAI, Inc., 2024] orchestrates role-playing autonomous agents that collaborate through configurable process models and event-driven flows, emphasizing multi-agent coordination over the layered safety and budget controls that we prioritize. smolagents [Roucher et al., 2025], Hugging Face's minimalist agent library, has its CodeAgent write actions as executable Python snippets, reducing step counts by 30%; however, it does not address repository-level concerns such as git isolation or continuation across sessions. The OpenAI Agents SDK [OpenAI, 2025c] offers a provider-agnostic framework for multi-agent workflows with handoffs, guardrails, and tracing, while Google's Agent Development Kit [Google, 2025b] provides an open-source toolkit for building, evaluating, and deploying AI agents with multi-agent orchestration. Both provide general-purpose agent infrastructure but leave domain-specific concerns to the application layer. Our architecture is purpose-built for software engineering and general research, with each layer addressing a specific practical concern (budget tracking, continuation, code safety).

LLM agent surveys. Several comprehensive surveys have mapped the rapidly growing landscape of LLM-based agents. Xi et al. [2023] surveys the design space of LLM agents along three dimensions — brain (reasoning), perception (input modalities), and action (tool use) — and catalogs applications across social science, natural science, and engineering. Wang et al. [2024a] propose a systematic framework for autonomous agents built on LLMs, covering profile, memory, planning, and action modules. Our system can be understood through the lens of these frameworks: the KISS Agent implements the action loop; the Relentless Agent addresses memory and planning concerns through continuation summaries; and the system prompt encodes the profile.

IDE integration. GitHub Copilot [GitHub, 2021], Cursor [Cursor, 2024], and Windsurf [Codeium, 2024] provide AI assistance within editors. Cursor recently released Composer 2 [Cursor Research, 2026], a custom fine-tuned coding model trained with large-scale reinforcement learning that achieves 61.7% on Terminal Bench 2.0. We operate at the level of autonomous multi-step task execution with full tool access and git-level isolation.

8.1 Dynamic Steering of Running Agents

A practical autonomous assistant must remain steerable after a task has started. KISS Sorcar therefore supports *dynamic steering*: while any agent is running, the user can add a new natural-language message to the live task, and that message is delivered to the selected agent as additional user guidance. The agent does not need to finish its current high-level task, reach an explicit permission prompt, or be restarted from a new initial prompt. The mechanism is intentionally simple: a running task remains a conversation, and the user's later message becomes part of the task trajectory that the agent must incorporate when it next reaches a safe decision point. This is useful when the user observes that the agent has misunderstood a requirement, discovers a new constraint, notices that an external command is taking too long, or wants to redirect a parallel exploration toward a more promising branch.

This feature is orthogonal to the continuation mechanism in Section 2.2. Relentless continuation preserves progress across context-window or step-budget boundaries; dynamic steering lets the human change the objective while progress is still being made. It is also orthogonal to worktree isolation: steering can redirect the behavior of a task without sacrificing the ability to later commit-and-merge or discard the whole branch. Because KISS Sorcar treats external systems such as Claude Code CLI and Codex CLI as model backends and exposes third-party messaging agents through the same tool layer, dynamic steering is not tied to a single first-party agent. The same user action can steer a local KISS Agent, a worktree-isolated Sorcar task, or an agent that delegates work through a supported external backend.

Several recent systems provide related forms of in-progress control. Claude Code's Remote Control allows a user to continue a local session from a browser or phone, keep the conversation synchronized across devices, and send messages interchangeably from terminal, web, and mobile surfaces [Anthropic, 2026c]. This is the closest product analogue: it explicitly targets steering in-progress work from another device. KISS Sorcar differs in scope: dynamic steering is an IDE/runtime primitive for any KISS-managed agent, including third-party backends and parallel sub-agents, rather than a remote

surface for one vendor's agent. GitHub Copilot's cloud agent can be monitored and steered with follow-up prompts in GitHub, and users can mention `@copilot` on a pull request to request further changes [GitHub, 2026]. However, issue comments added after initial assignment are not observed until the workflow moves to the pull request, and follow-ups often create or continue PR-bound sessions. KISS Sorcar instead treats steering as a live message to the active task, independent of the GitHub issue/PR lifecycle.

Cursor Cloud Agents and OpenAI Codex both expose richer background-agent workflows. Cursor can launch cloud agents from web, desktop, Slack, GitHub/Bitbucket comments, Linear, and an API, and it provides artifacts plus remote desktop control of the agent's environment [Cursor, 2026]. Codex has added mobile remote workflows, queue-versus-steer follow-up behavior, side chats, and progress visibility for running threads and subagents [OpenAI, 2026a]. These systems demonstrate that users need to follow and redirect long-running agent work. KISS Sorcar's contribution is to make that capability small, uniform, and backend-agnostic: the user's steering message is routed through the same local task machinery as the original instruction, rather than through a product-specific cloud session, remote desktop handoff, or PR comment loop.

Other related mechanisms are more specialized. Aider's `-watch-files` mode lets users place `AI!` or `AI?` comments in source files, which Aider collects as coding instructions from the editor [Gauthier, 2026]. This provides in-context steering through file edits, but it is specific to Aider and to source-code comments. LangChain/LangGraph's human-in-the-loop middleware pauses execution around configured tool calls and resumes after a human approves, edits, rejects, or responds to the pending action [LangChain, 2026]. Semantic Kernel similarly describes the agent loop as repeated function calling until completion or until the model needs user help [Microsoft, 2025]. These frameworks provide useful interrupt/resume substrates, but they center on tool-call approval or model-requested help. Dynamic steering in KISS Sorcar is broader: the user may inject arbitrary revised task guidance even when the agent did not explicitly ask for input, allowing human intent to remain in the loop throughout long-running autonomous work.

Recent advances in agentic software engineering. The field has matured rapidly since late 2025. Hassan et al. [2025] articulate the foundational pillars of *Agentic Software Engineering* (SE 3.0), defining a research roadmap that emphasizes trust, controllability, and goal-oriented task decomposition. Li et al. [2025] surveys the landscape of autonomous coding agents and characterizes the transition from assistive code completion to full-fledged AI teammates that initiate, plan, and execute development tasks. We instantiate several of the principles advocated in these roadmaps, including layered controllability (via budget tracking and step limits) and safe isolation (via worktrees).

Wang et al. [2025] provides a comprehensive survey of AI agentic programming techniques, cataloging how LLM-based agents decompose goals, interact with compilers and version control systems, and self-correct through feedback loops. Chatlatanagulchai et al. [2025] study *agent context files* — persistent, project-level instruction files that guide agentic coding tools — and find that high-quality context files significantly improve agent performance. Our layered system prompt and `SORCAR.md` override mechanisms are instances of this pattern. Mohsenimofidi et al. [2025] further investigates context engineering for AI agents in open-source software, highlighting how curated context improves agent efficacy on repository-level tasks.

Evolving benchmarks for coding agents. SWE-bench Pro [Deng et al., 2025] introduces a substantially more challenging benchmark with 1,865 long-horizon problems drawn from 41 repositories, including commercial codebases, explicitly targeting enterprise-level complexity beyond the original SWE-bench. These long-horizon tasks — often requiring multi-file patches and hours of professional developer effort — directly motivate our continuation mechanism. Prathifkumar et al. [2025] raises the important question of benchmark contamination, showing that overlap between SWE-Bench-Verified problems and LLM training data may inflate scores, suggesting that high benchmark numbers may partly reflect memorization rather than genuine problem-solving ability. Horikawa et al. [2025] provides an empirical study of how AI coding agents handle refactoring tasks, finding that while agents can plan and execute complex refactorings, they still struggle with cross-file dependency analysis — a challenge that our sub-agent parallelism partially addresses.

Self-improvement and test-time scaling. Robeyns et al. [2025] demonstrates a self-improving coding agent that iteratively refines its own scaffolding code, achieving dramatic benchmark improvements through self-generated optimizations. We previously experimented with a much lighter-weight variant of this idea via a per-project preferences file that the agent read and updated each session,

but removed the mechanism (Section 6.8) because the store retained stale project facts that went out of date as the code evolved, leading the agent to reintroduce already-fixed bugs. Gao et al. [2025] introduces the Trae Agent, which applies test-time compute scaling to software engineering, dynamically allocating more inference budget to harder problems. Our budget-tracking mechanism provides a complementary perspective: rather than scaling compute adaptively, we enforce hard budget ceilings while the Relentless Agent ensures maximum progress within those bounds.

Community-driven prompt engineering. Get Shit Done (GSD) [TÂCHES, 2025] is a light-weight meta-prompting, context engineering, and spec-driven development system originally created for Claude Code and later extended to other AI coding agents. GSD addresses *context rot*—the quality degradation that occurs as an LLM fills its context window—by structuring work into phased planning documents and spawning specialized parallel agents for research, planning, execution, and verification. Its core philosophy rejects enterprise ceremony in favor of directness: "no enterprise roleplay bullshit," as the author puts it. The sections "Pre-flight Checks", "Deep Work", and "Pre-Finish Verification" of `SYSTEM.md` were partly inspired by this work.

9 Conclusion

We have shown that a simple layered, single-concern architecture can address the practical challenges of deploying LLM agents for real-world software development. On Terminal Bench 2.0, a benchmark of 89 diverse terminal-based tasks evaluated across 5 trials each, our system achieves a 62.2% overall pass rate (277/445), compared with Claude Code (58%) and Cursor Composer 2 (61.7%) on the same benchmark with the same underlying model (Claude Opus 4.6). These results are achieved without benchmark-specific optimizations, fine-tuning, or reinforcement learning.

We complement the architecture with a system prompt that encodes engineering practices directly into the agent's behavior: read before writing, test before fixing, plan before executing, verify before finishing. These are not novel insights—they are the practices of careful software engineering, translated into instructions that an LLM can follow. The evaluation suggests that giving a frontier model the time and tools to validate its own output matters more than model-level customization.

In summary, we showed that a simple agent framework, without sophisticated agent technologies such as trajectory compaction and asynchronous multi-agent orchestration, was sufficient to build KISS Sorcar. By building KISS Sorcar using itself and matching or exceeding both Cursor and Claude Code, we found that established software engineering techniques and principles are important for building reliable agent systems.

An Important Advice

We found that researchers are publishing lots of papers on AI, and it is hard to keep track of them or validate their claims. We propose to use the following prompt with KISS Sorcar to identify issues in blogs, papers, and code repositories:

```
Can you read <<url>>, and thoroughly and precisely check for **wrong assumptions**, **cheating**,
**irreproducibility issues**, **fraud**, **potential for cheating in evaluation**,
**AI slop**, and **security vulnerabilities**?
Use the internet search extensively and do not believe what people say--verify it yourself.
Do not hesitate to download the code and run it to validate the results.
For security vulnerabilities, create a POC and test it.
Generate an HTML report in ./sorcar_reported_frauds/ and open it in the user's default browser.
Thoroughly fact check everything you claim in the report.
```

In the prompt, replace «url» with the actual link to a blog, a paper, or a code repository.

Acknowledgments

This research is supported in part by gifts from Accenture, Amazon, AMD, Anyscale, Broadcom, Google, IBM, Intel, Intesa Sanpaolo, Lambda, Lightspeed, Mibura, NVIDIA, Samsung SDS, SAP, by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research through the X-STACK: Programming Environments for Scientific Computing program

(DESC0021982,) and the Defense Advanced Research Projects Agency (DARPA) under Agreement No. HR00112590134.

We would like to thank Marius Momeu for finding critical bugs in the cost computation and in the UI usability, Yogya Mehrotra for testing and fixing bugs in the OpenClaw like third_party_agents in KISS Sorcar; Debabrata Dash, Yiwei Hou, Kaiyao Ke, Muxi Lyu, Anoop Mishra, Manish Shetty, Ion Stoica, Shangyin Tan, Hao Wang, and Matei Zaharia for various useful feedback.

References

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2026. Oral. arXiv preprint arXiv:2507.19457.

Algorithmic Superintelligence. OpenEvolve: Open-source implementation of AlphaEvolve. `https://github.com/algorithmicsuperintelligence/openevolve`, 2025.

Anthropic. Anthropic prompt engineering guide. `https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview`, 2025a.

Anthropic. Claude Code: Anthropic's agentic coding system. `https://www.anthropic.com/product/claude-code`, 2025b.

Anthropic. Introducing Claude Opus 4.6. `https://www.anthropic.com/news/claude-opus-4-6`, 2026a.

Anthropic. Introducing Claude Opus 4.7. `https://www.anthropic.com/news/claude-opus-4-7`, 2026b.

Anthropic. Claude Code Remote Control: Continue local sessions from any device. `https://docs.anthropic.com/en/docs/claude-code/remote-control`, 2026c.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Worawalan Chatlatanagulchai, Hao Li, Yutaro Kashiwa, Brittany Reid, Kundjanasith Thonglek, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, Bram Adams, Ahmed E. Hassan, and Hajimu Iida. Agent READMEs: An empirical study of context files for agentic coding. *arXiv preprint arXiv:2511.12884*, 2025.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Audrey Cheng, Shu Liu, Melissa Pan, Zhifei Li, Bowen Wang, Alex Krentsel, Tian Xia, Mert Cemri, Jongseok Park, Shuo Yang, Jeff Chen, Lakshya Agrawal, Aditya Desai, Jiarong Xing, Koushik Sen, Matei Zaharia, and Ion Stoica. Barbarians at the gate: How AI is upending systems research. *arXiv preprint arXiv:2510.06189*, 2025.

Codeium. Windsurf: The AI-powered IDE. `https://windsurf.com`, 2024.

Cognition Labs. Devin: The first AI software engineer. `https://devin.ai`, 2024.

CrewAI, Inc. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. `https://github.com/crewAIInc/crewAI`, 2024.

Cursor. Cursor: The AI-first code editor. `https://cursor.sh`, 2024.

Cursor. Cursor cloud agents. `https://cursor.com/docs/cloud-agent`, 2026.

Cursor Research. Composer 2 technical report. *arXiv preprint arXiv:2603.24477*, 2026.

Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, et al. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.

Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*, 2023.

Pengfei Gao, Zhao Tian, Xiangxin Meng, and Trae Research Team. Trae agent: An LLM-based agent for software engineering with test-time scaling. *arXiv preprint arXiv:2507.23370*, 2025.

Paul Gauthier. Aider: AI pair programming in your terminal. `https://github.com/paul-gauthier/aider`, 2023.

Paul Gauthier. Aider in your IDE: AI comments and file watching. `https://aider.chat/docs/usage/watch.html`, 2026.

GitHub. GitHub Copilot: Your AI pair programmer. `https://github.com/features/copilot`, 2021.

GitHub. Using Copilot cloud agent on GitHub. `https://docs.github.com/en/copilot/how-tos/use-copilot-agents/cloud-agent/use-cloud-agent-on-github`, 2026.

Google. Gemini prompt engineering strategies. `https://ai.google.dev/gemini-api/docs/prompting-strategies`, 2025a.

Google. Agent Development Kit (ADK): An open-source framework for building AI agents. `https://google.github.io/adk-docs/`, 2025b.

Google DeepMind. Gemini 3.1 Pro. `https://deepmind.google/models/gemini/pro/`, 2026.

Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yun Xiong, and Wenfeng Liang. DeepSeek-Coder: When the large language model meets programming — the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.

Ahmed E. Hassan, Hao Li, Dayi Lin, Bram Adams, Tse-Hsun Chen, Yutaro Kashiwa, and Dong Qiu. Agentic software engineering: Foundational pillars and a research roadmap. *arXiv preprint arXiv:2509.06216*, 2025.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.

Kosei Horikawa, Hao Li, Yutaro Kashiwa, Bram Adams, Hajimu Iida, and Ahmed E. Hassan. Agentic refactoring: An empirical study of AI coding agents. *arXiv preprint arXiv:2511.04824*, 2025.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In *The Twelfth International Conference on Learning Representations*, 2024.

Kimi Team. Kimi K2.5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.

LangChain. LangChain: Build context-aware reasoning applications. `https://github.com/langchain-ai/langchain`, 2022.

LangChain. Human-in-the-loop middleware. `https://docs.langchain.com/oss/python/langchain/human-in-the-loop`, 2026.

Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. ShinkaEvolve: Towards open-ended and sample-efficient program evolution. *arXiv preprint arXiv:2509.19349*, 2025.

Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. The rise of AI teammates in software engineering (SE 3.0): How autonomous coding agents are reshaping software engineering. *arXiv preprint arXiv:2507.15003*, 2025.

Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. StarCoder: May the source be with you! *Transactions on Machine Learning Research (TMLR)*, 2023.

Microsoft. Planning in Semantic Kernel. `https://learn.microsoft.com/en-us/semantic-kernel/concepts/planning`, 2025.

Seyedmoein Mohsenimofidi, Matthias Galster, Christoph Treude, and Sebastian Baltes. Context engineering for AI agents in open-source software. *arXiv preprint arXiv:2510.21413*, 2025.

Yohei Nakajima. BabyAGI. `https://github.com/yoheinakajima/babyagi`, 2023.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

OpenAI. Introducing Codex: A cloud-based software engineering agent. `https://openai.com/index/introducing-codex/`, 2025a.

OpenAI. Prompt engineering best practices. `https://platform.openai.com/docs/guides/prompt-engineering`, 2025b.

OpenAI. OpenAI Agents SDK: A lightweight, powerful framework for multi-agent workflows. `https://github.com/openai/openai-agents-python`, 2025c.

OpenAI. Codex changelog. `https://developers.openai.com/codex/changelog/`, 2026a.

OpenAI. Introducing GPT-5.5. `https://openai.com/index/introducing-gpt-5-5/`, 2026b.

OpenClaw AI. OpenClaw: Personal AI assistant. `https://openclaw.ai`, 2025.

Krista Opsahl-Ong, Michael J. Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024. arXiv preprint arXiv:2406.11695.

Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs write efficient GPU kernels? *arXiv preprint arXiv:2502.10517*, 2025.

Thanosan Prathifkumar, Noble Saji Mathews, and Meiyappan Nagappan. Does SWE-Bench-Verified test agent ability or model memory? *arXiv preprint arXiv:2512.10218*, 2025.

Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. Automatic prompt optimization with "gradient descent" and beam search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. arXiv preprint arXiv:2305.03495.

Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent. *arXiv preprint arXiv:2504.15228*, 2025.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024. doi: 10.1038/s41586-023-06924-6.

Aymeric Roucher, Albert Villanova del Moral, Thomas Wolf, Leandro von Werra, and Erik Kaunismäki. smolagents: A smol library to build great agentic systems. `https://github.com/huggingface/smolagents`, 2025.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Significant Gravitas. AutoGPT. https://github.com/Significant-Gravitas/AutoGPT, 2023.

Adam Stein, Davis Brown, Hamed Hassani, Mayur Naik, and Eric Wong. Finding widespread cheating on popular agent benchmarks. Blog post, https://debugml.github.io/cheating-agents/, 2026a.

Adam Stein, Davis Brown, Hamed Hassani, Mayur Naik, and Eric Wong. Detecting safety violations across many agent traces. *arXiv preprint arXiv:2604.11806*, 2026b.

TÂCHES. Get Shit Done: A light-weight meta-prompting, context engineering and spec-driven development system for AI coding agents. https://github.com/gsd-build/get-shit-done, 2025. Initial commit December 2025.

Hao Wang, Qiuyang Mang, Alvin Cheung, Koushik Sen, and Dawn Song. We scored 100% on AI benchmarks without solving a single problem. Blog post, https://moogician.github.io/blog/2026/trustworthy-benchmarks/, 2026.

Huanting Wang, Jingzhi Gong, Huawei Zhang, Jie Xu, and Zheng Wang. AI agentic programming: A survey of techniques, challenges, and opportunities. *arXiv preprint arXiv:2508.11126*, 2025.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024a.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhi Li, Hao Peng, and Heng Ji. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024b.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations (ICLR)*, 2024a. arXiv preprint arXiv:2309.03409.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024b.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023a.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023b.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. TextGrad: Automatic "differentiation" via text. *arXiv preprint arXiv:2406.07496*, 2024.

Z.ai. GLM-5.1: Towards long-horizon tasks. Technical blog, https://z.ai/blog/glm-5.1, 2026.